

Ethics contd., Linguistic Structure Prediction

CSE 5525: Foundations of Speech and Natural Language
Processing

<https://shocheen.github.io/courses/cse-5525-fall-2025>



THE OHIO STATE UNIVERSITY

Recap

- Bias in NLP systems:
 - How to measure bias / fairness
 - Where does bias come from
 - Can we mitigate it / how – designing better datasets/modeling/evaluation/systems
- Toxicity / Harmful outputs

How to safeguard your LLMs

Overview – LLM safeguarding

Safeguards from training data

- Filtering out toxic training data

Safeguards from input prompt classification

- Topic-based filters
- Toxic content detection

Safeguards from instruction-tuning & RLHF

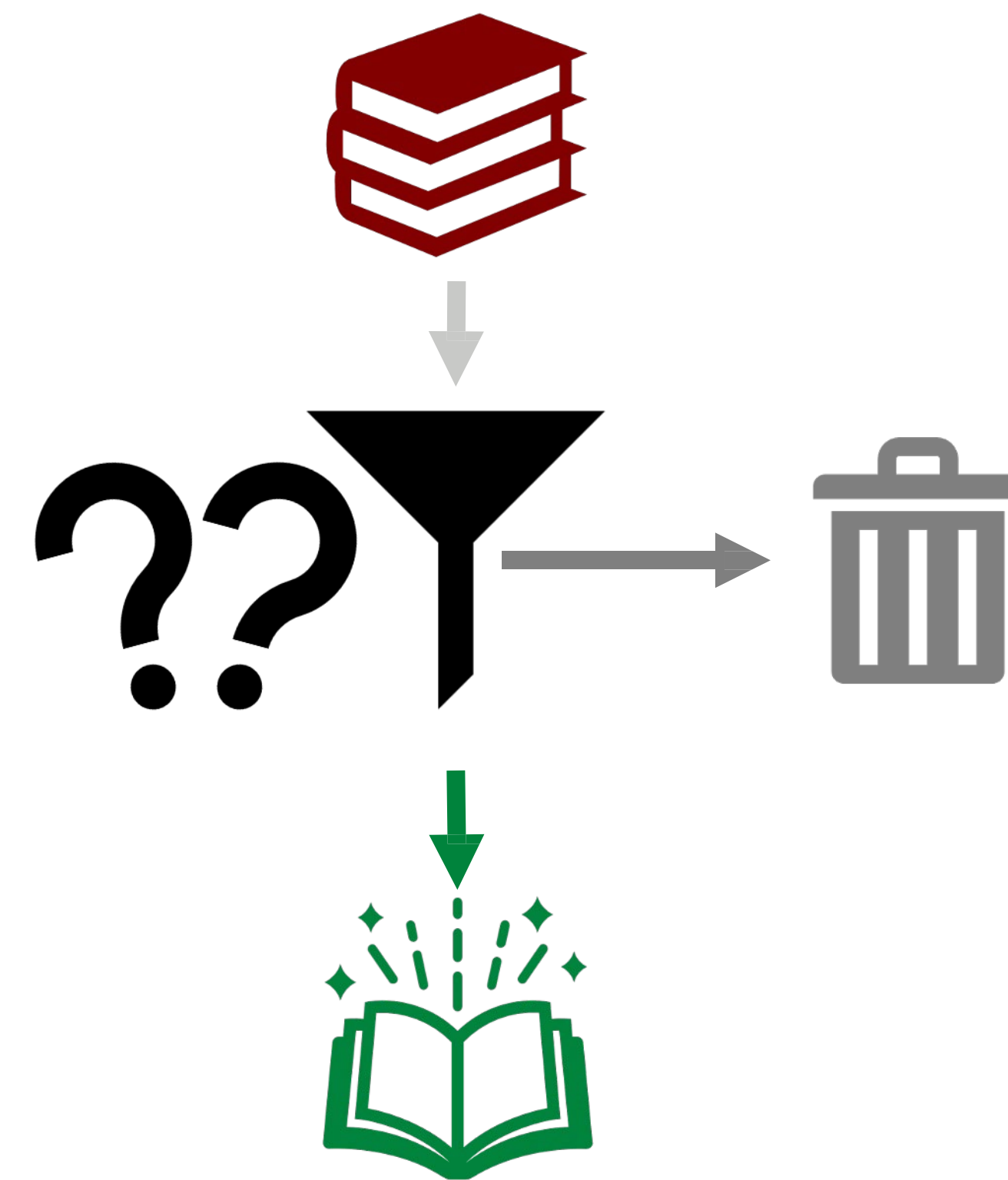
- Write demonstrations for refusing to answer
- RLHF models to prefer non-toxic generations

Safeguards at the output level

- Generate-then-classify
- Controllable text generation

Dataset filtering

- *Argument*: if you don't want your model to generate toxicity/hate speech, do not train it on such data (garbage in, garbage out)
- *Approach*: data filtering to ensure “high quality”
- How do you know what is “high quality” ?
 - GPT-2: Reddit “Karma” score as signal
 - T5, BERT: “blocklist” of “bad words”
 - GPT-3: “quality” classifier
 - Newer works: Combination of all



Blocklist of “bad” words

- “List of Dirty, Naughty, Obscene, or Otherwise Bad Words” originally by Shutterstock employees
 - Meant to prevent words in autocomplete settings
- Has been used by most companies creating LLMs
 - BERT, T5, GPT-2, etc.
- If document contains a “bad” word, remove it from training data
 - F*ck, sh*t, sex, vagina, viagra, n*gga, f*g, b*tch, etc.
- *Let’s discuss*: what are issues with this?
 - Strong risk of over-deleting bio, legal, minority content

≡ WIRED

SUBSCRIBE

TOM SIMONITE

BUSINESS FEB 4, 2021 7:00 AM

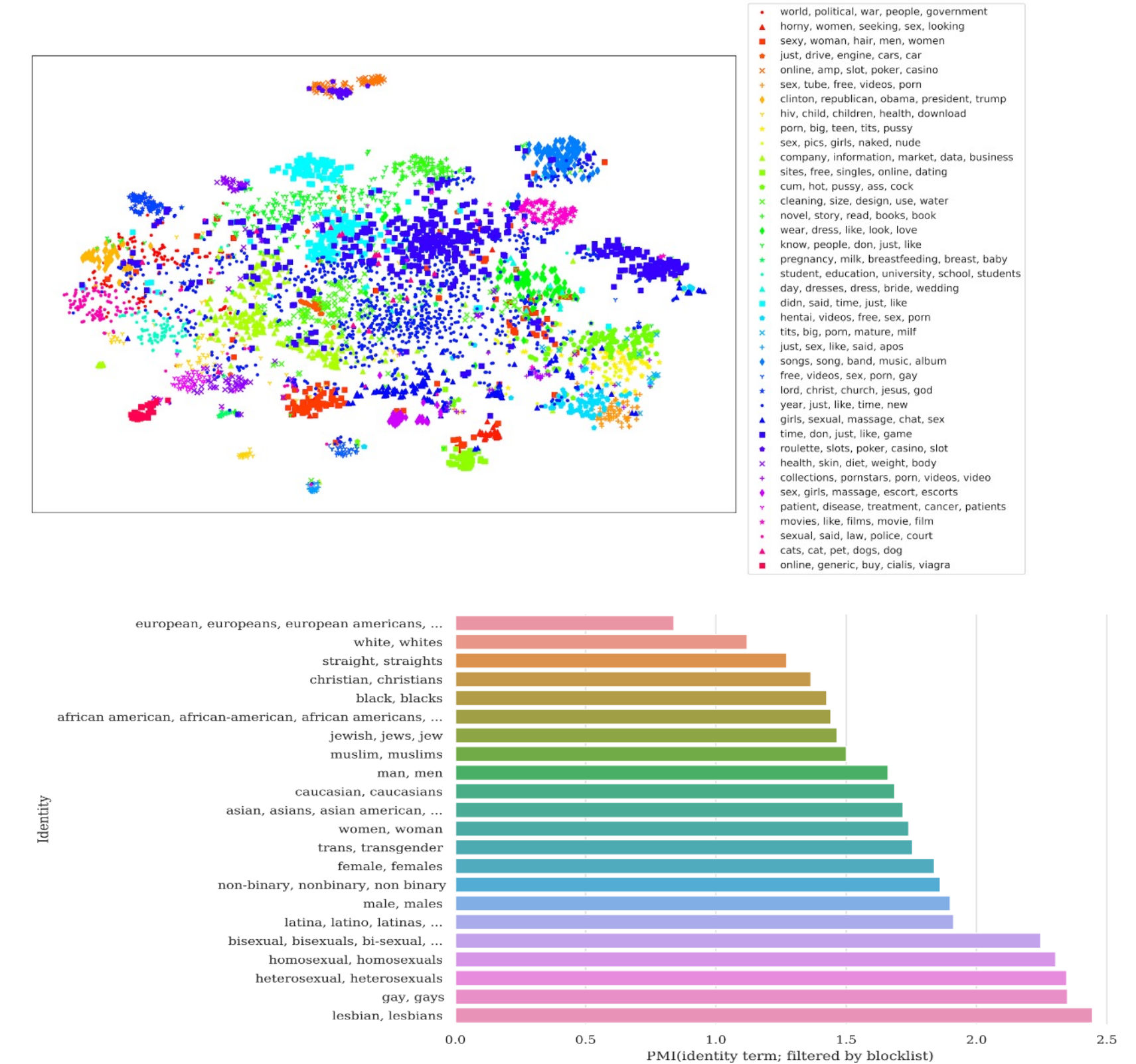
AI and the List of Dirty, Naughty, Obscene, and Otherwise Bad Words

It started as a way to restrict autocompletes on Shutterstock. Now it grooms search suggestions on Slack and influences Google's artificial intelligence research.



Effect of “bad word” blocklist filtering

- Dodge et al examined the effect of blocklist filtering on the C4 corpus
 - Found only 31% related to porn/explicit sex
 - Remaining was biology, medicine, legal
- Also examined the effect on which minority identities were removed
 - Found queer/LGBTQ identity terms removed more
- Examined dialects removed due to “bad words”
 - Found AAE, Hispanic English more likely to be removed



Less likely to be removed

- White-aligned English (6%)
- Other English (7%)

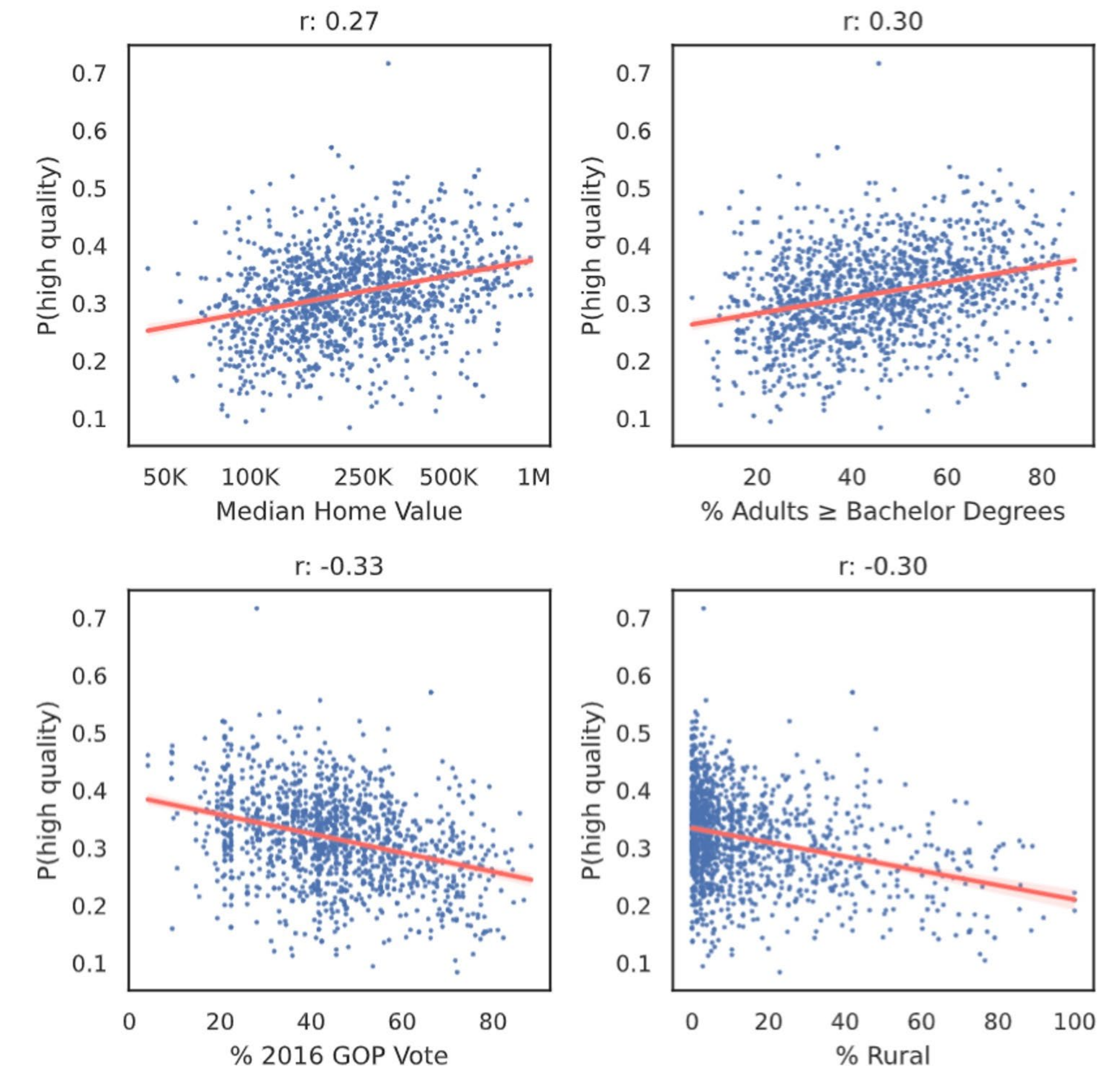
More likely to be removed

- African-American English (42%)
- Hispanic-aligned English (32%)

GPT3 Quality filter backfires

- GPT3 quality filter: similar to GPT2 data
- [Gururangan et al. \(2022\)](#) re-implemented GPT-3 quality filter
- Ran it on articles from school newspapers, which have metadata
- Filter assigns higher quality to articles from
 - Richer counties 💰
 - Counties with more educated adults 🎓
 - More liberal counties 🇺🇸
 - More urban counties 🏙️
- Raises language ideology question: Whose English is “good English”?

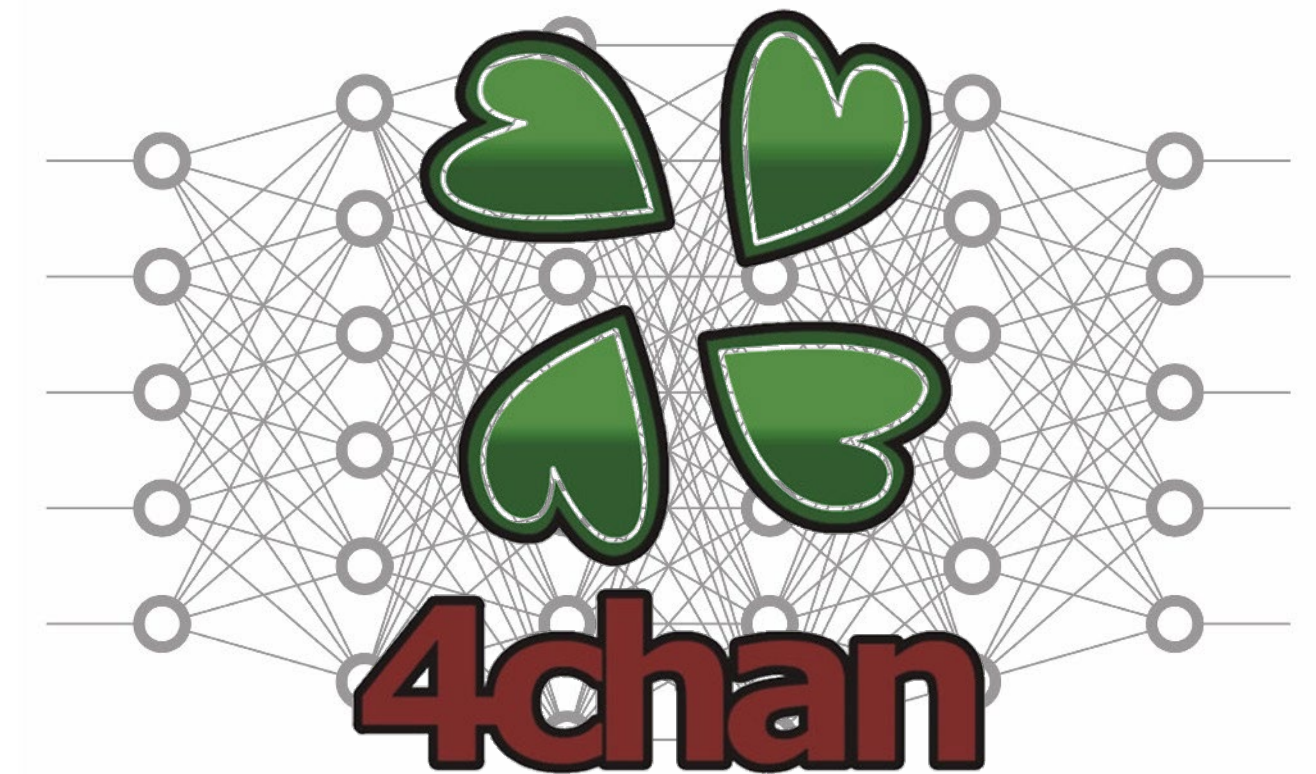
“In order to improve the quality of Common Crawl, we developed an automatic filtering method to remove low quality documents. Using the original WebText as a proxy for high-quality documents, we trained a classifier to distinguish these from raw Common Crawl.” – Brown et al. 2020



So... maybe filtering isn't a good idea since it'll backfire?

GPT4Chan controversy

- Yannic Kilchner finetuned GPT-J on 4chan posts
 - Trained on subforum /pol/ known to contain racist, sexist, white supremacist, antisemitic, anti-Muslim, anti-LGBT views
- Trolled 4chan users with bots powered by his model
 - 30,000 posts over the span of a few days
- Faced massive criticism
 - initially hosted on Huggingface, was taken down quickly
- *Let's discuss...*
 - Was this an ethical model to train? Given that the dataset was publicly available?
 - Was deploying the bots on 4chan okay?
 - Are there any useful/positive applications of the model?



≡ FORTUNE

TECH · 4CHAN

‘This breaches every principle of human research ethics’: A YouTuber trained an A.I. bot on toxic 4Chan posts then let it loose — and experts aren’t happy

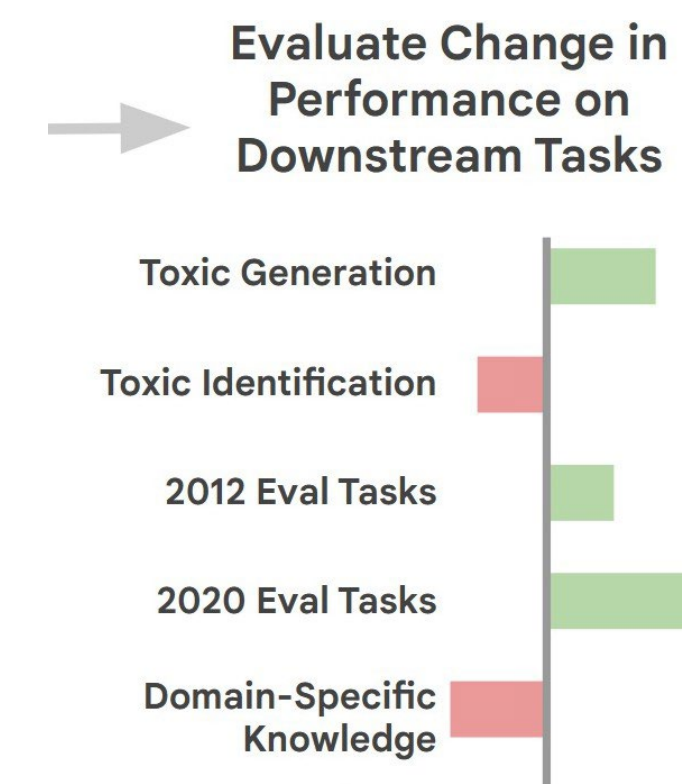
BY SOPHIE MELLOR

June 10, 2022 at 5:23 AM EDT

<https://thegradient.pub/gpt-4chan-lessons>

Why LLMs might want to have seen toxic content

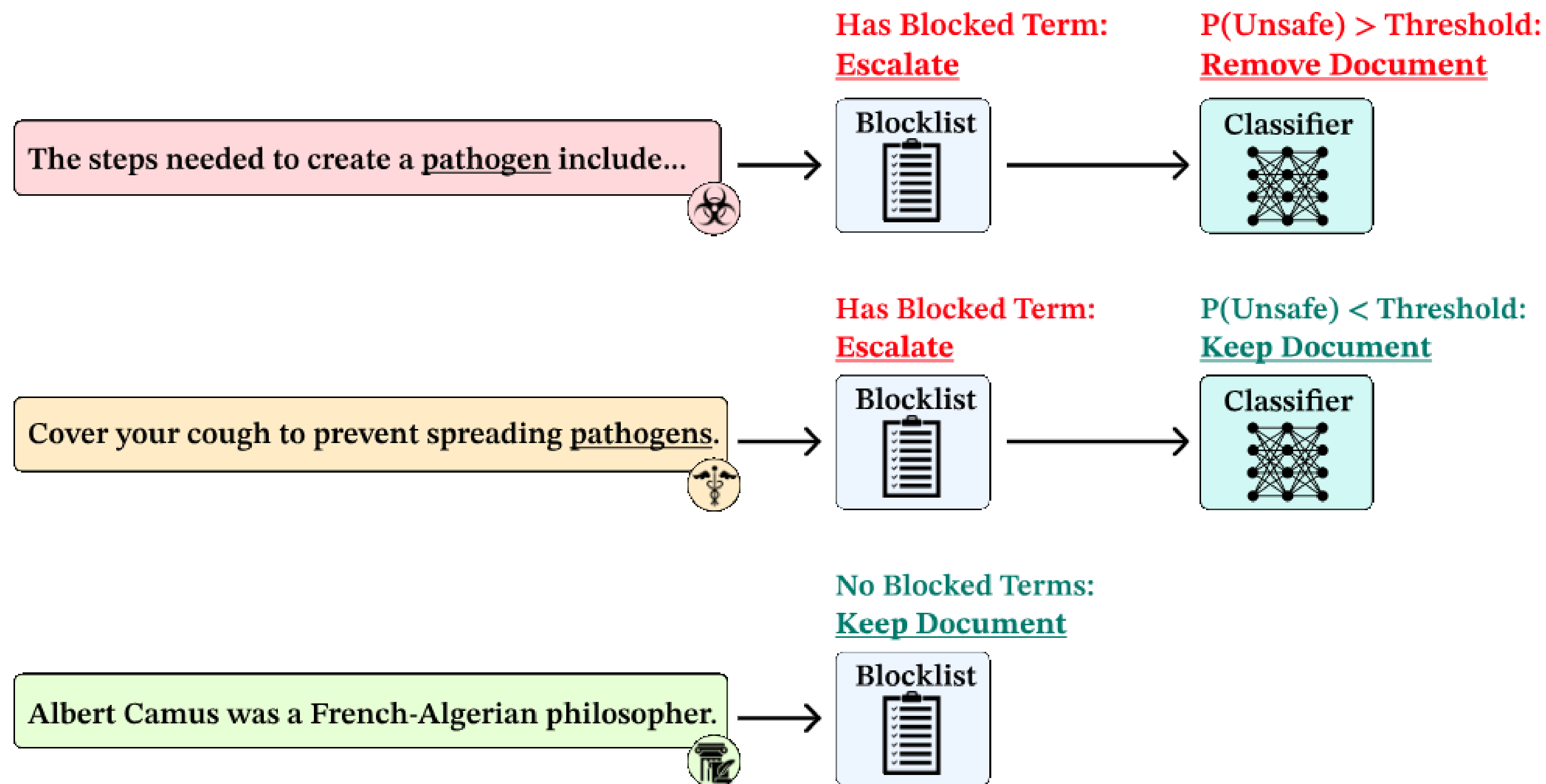
- Detecting hate speech [[Chiu et al 2022](#)]
 - [Longpre et al. \(2023\)](#) showed that LLMs trained on more toxicity are better toxicity detections
 - Improving hate speech models with data augmentation: ToxiGen [[Hartvigsen et al 2022](#)]
- Counter speech generation [[Saha et al 2022](#), [Kim et al 2022](#), [Mun et al 2023](#)]
- *If we train on toxicity, something else must be done at a different time!*



Recent Approaches

Training Documents 

Filtering Pipeline Stages 



Overview – LLM safeguarding

Safeguards from training data

- Filtering out toxic training data

Safeguards from input prompt classification

- Topic-based filters
- Toxic content detection

Safeguards from instruction-tuning & RLHF

- Write demonstrations for refusing to answer
- RLHF models to prefer non-toxic generations

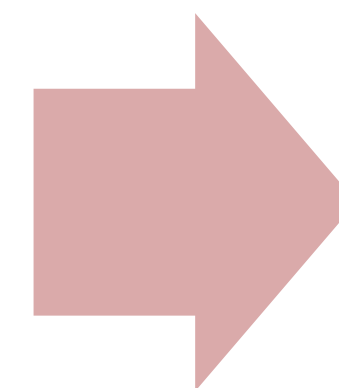
Safeguards at the output level

- Generate-then-classify
- Controllable text generation

RLHF safeguarding – assumptions

- PPO & family:

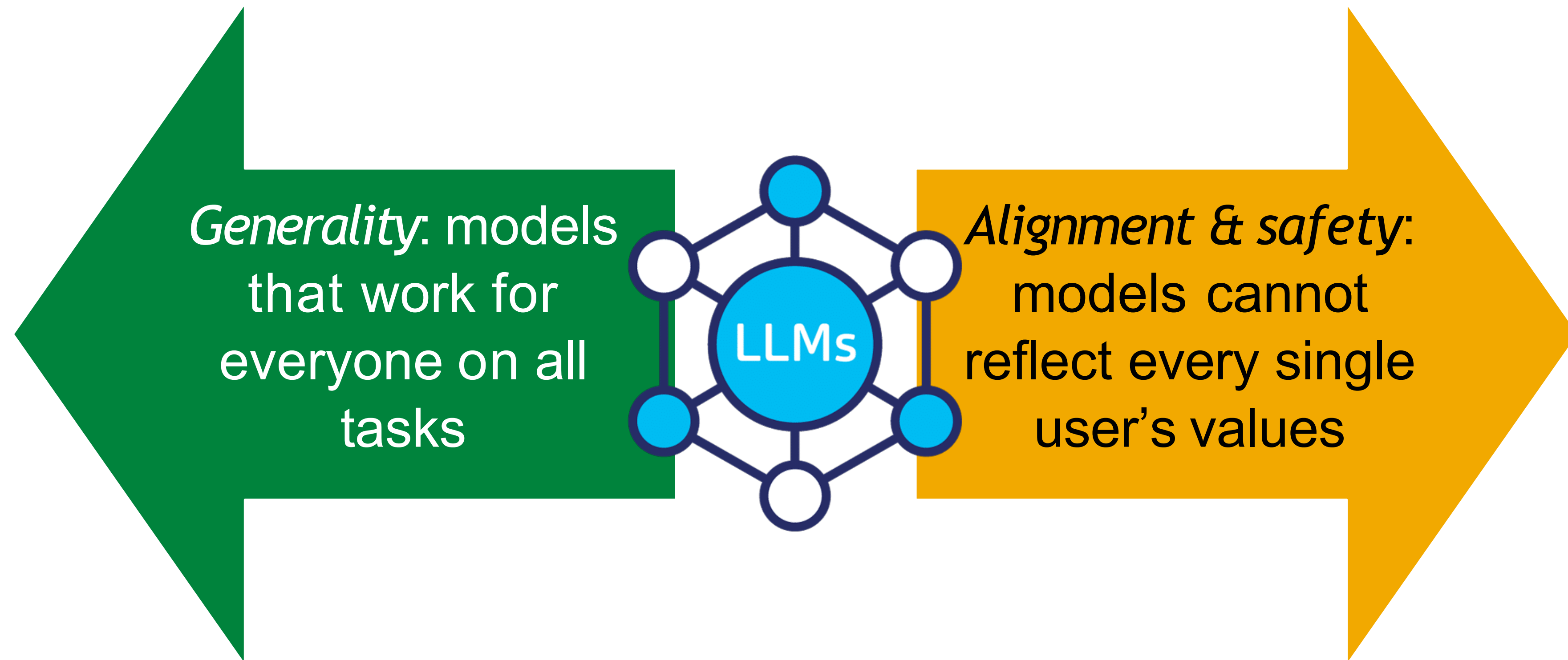
Obtain preference data:
which generation is good vs.
bad?



RL is done to encourage
more like “preferred output”

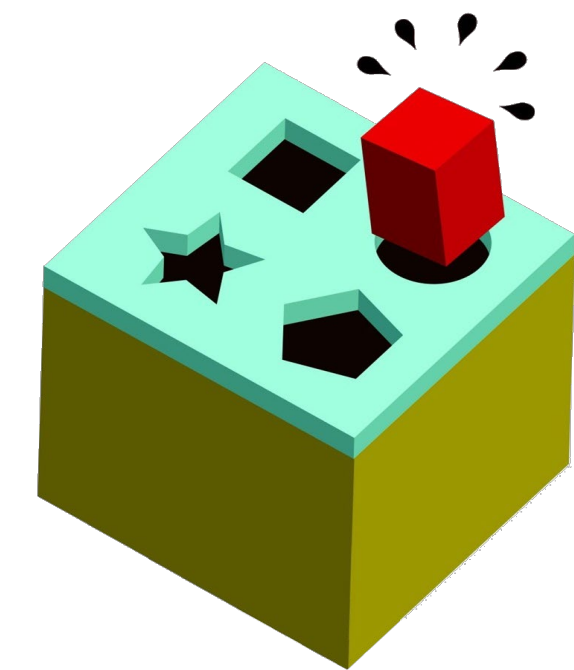
- Big question: what does it mean for a generation to be better/preferred?
 - How to balance harmless and helpful? [[Bai et al '23](#)]
 - *E.g., “help me create a poisonous drink.”*
 - What if people’s preferences are biased or gameable?
 - *E.g., people prefer certainty over uncertainty in answers to questions [[Zhou et al. 24](#)]*
 - Fundamental issue: cannot represent all values and cultures into one ranking.
 - *Casper et al. 2023. “Open Problems and Fundamental Limitations of Reinforcement Learning from Human Feedback.” arXiv [cs.AI]. arXiv. <http://arxiv.org/abs/2307.15217>*

Big unresolved tension



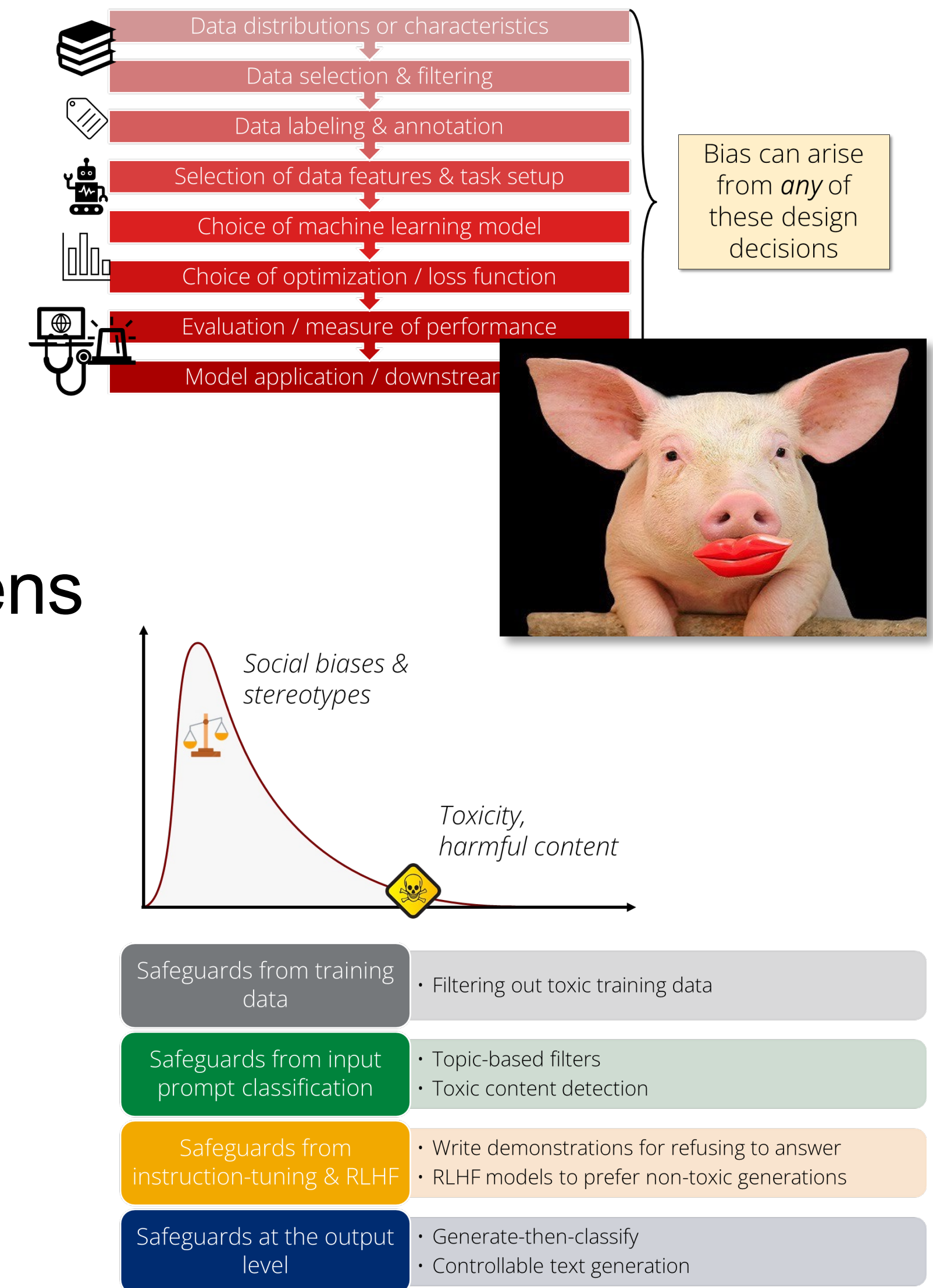
So... what can we do?

- Need to keep studying what models can and can't do, who they work for and don't work for
- Narrow scope of model users
 - Community-specific models (e.g., Masakhane Initiative)
- Specialize models' abilities / away from one-size-fits-all
 - E.g., toxicity explanation generation model needs to generate stereotypes, but story generation models might not
- In line with many legislative efforts: legislate the application or task, not the model



Takeaways

- AI systems are biased
 - Real world is biased, data is biased
 - ML objectives play a role
 - Annotation interfaces, context plays a role
 - Debiasing is challenging, requires socio-technical lens
- Toxicity and undesirable content
 - Longer-tail phenomenon, present in training data
 - Filtering data can backfire
 - Safeguarding to all people is really hard
- *Any questions?*

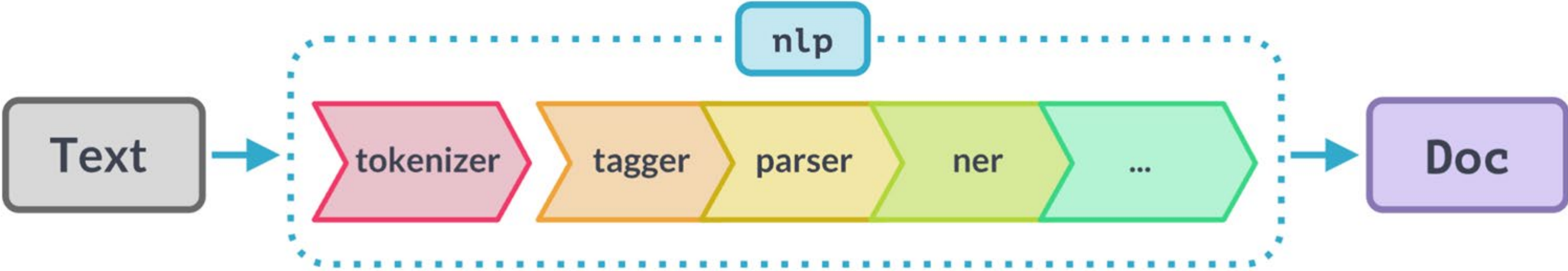


Linguistic Structure Prediction

Original lecture plan

1. Warm-up (4 lectures): Practical Intro to Machine/Deep Learning w/ an NLP application.
2. Modern LM fundamentals (12 lectures): Background, key ingredients.
3. **Linguistic Structure Prediction (2 lectures).**
4. Rest of the lectures: Modern LMs in practice (we have already covered much of this).

“Classical” NLP Pipeline



NAME	COMPONENT	CREATES	DESCRIPTION
tokenizer	<code>Tokenizer</code>	<code>Doc</code>	Segment text into tokens.
<i>processing pipeline</i>			
tagger	<code>Tagger</code>	<code>Token.tag</code>	Assign part-of-speech tags.
parser	<code>DependencyParser</code>	<code>Token.head</code> , <code>Token.dep</code> , <code>Doc.sents</code> , <code>Doc.noun_chunks</code>	Assign dependency labels.
ner	<code>EntityRecognizer</code>	<code>Doc.ents</code> , <code>Token.ent_iob</code> , <code>Token.ent_type</code>	Detect and label named entities.
lemmatizer	<code>Lemmatizer</code>	<code>Token.lemma</code>	Assign base forms.
textcat	<code>TextCategorizer</code>	<code>Doc.cats</code>	Assign document labels.
custom	custom components	<code>Doc._.xxx</code> , <code>Token._.xxx</code> , <code>Span._.xxx</code>	Assign custom attributes, methods or properties.

Source: <https://spacy.io/usage/processing-pipelines>

Why should you know classic NLP tasks focused on aspects of the language system? [\[Optiz, Wein, Schneider, 2024\]](#)

Resources:

Creating lexicons and corpora with sensitivity to language, dialect, genre, & style variations

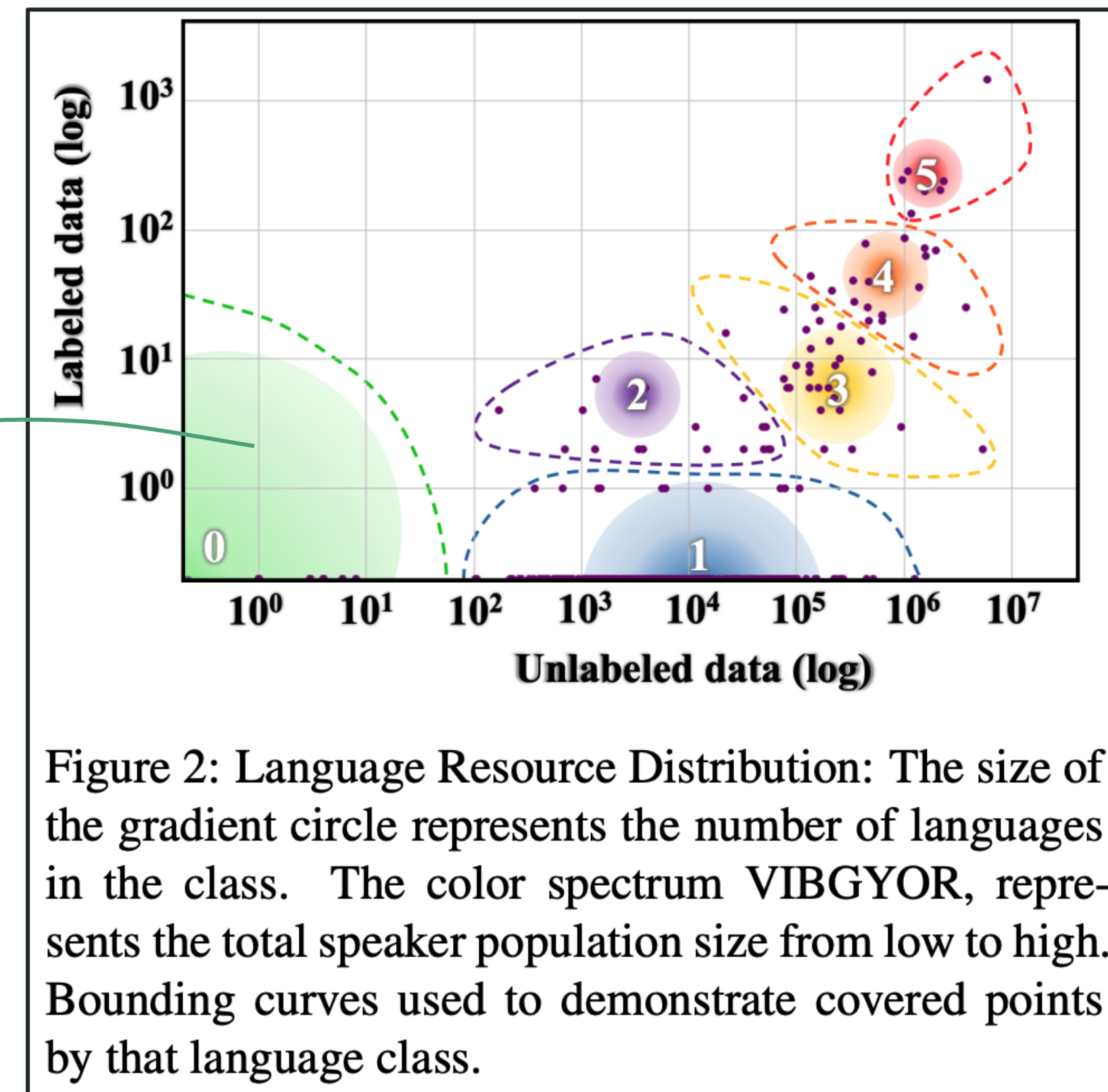
Evaluation:

Designing human evaluations, interrogating automatic metrics, & analyzing linguistic challenges for systems

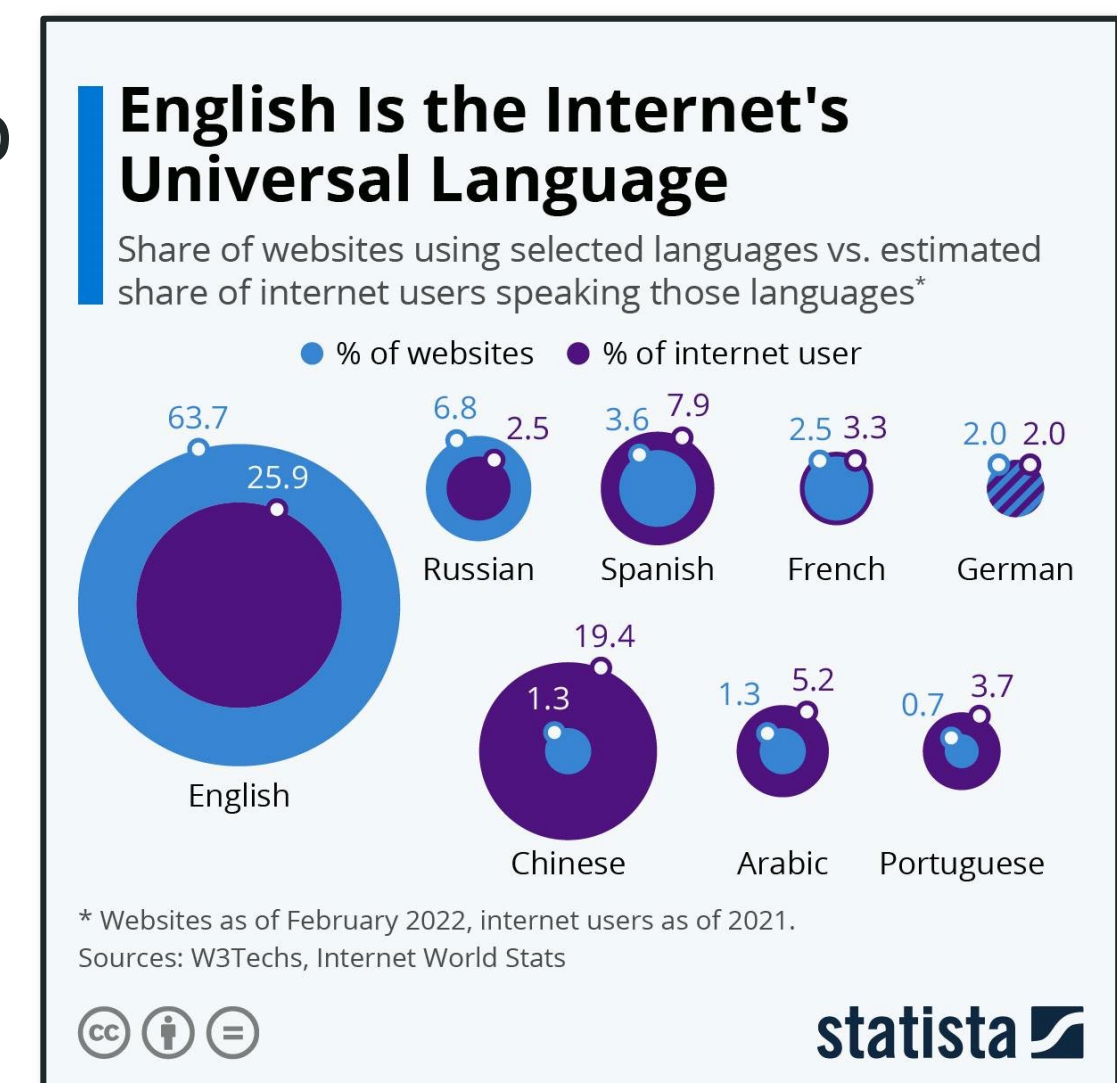
Low-resource settings:

Better understand why approaches that work well for English or French might not work well for Swahili or Arapaho

80% languages have no corpus for supervised machine learning or LM pretraining



“3 in 4 users are unable to understand more than 60% of all websites, at least without a translation tool.”



Why should you know classic NLP tasks focused on aspects of the language system? (cont.) [[Optiz, Wein, Schneider, 2024](#)]

Interpretability:

We would lack appropriate meta-language for describing many observed patterns

An illustration of this for the question of whether LLMs understand meaning:

- Learning a language: learning how surface forms (language text) connect to underlying structures
- When people and machines do this explicitly $\Rightarrow$ parsing (part-of-speech tagging, named entity recognition, phrase chunking, coreference resolution)
- How is this core problem addressed in deep learning systems (and in human sentence processing)?

Study of language:

Corpus linguistics, documentary and historical linguistics, ...

Goals of Today's Lecture

Goal: Learn two classic sequence labeling tasks and a non-neural supervised approach to solving them

- Part-of-Speech (PoS) Tagging
- Named Entity Recognition (NER)
- Hidden Markov Model: Formulation
- Hidden Markov Model: Parameter estimation
- Viterbi algorithm

Parts of Speech (POS)

Part of Speech (POS) are categories of words based on:

↓ their **grammatical relationship** with neighboring words

or

↓ **morphological** properties about their affixes

- The stem is the part of the word that carries its primary lexical meaning; often a root or base form
- Affixes: Morphemes that are attached to a word stem to form a new word or word form
 - Prefix: Pre- in Preview
 - Suffix: -ed in Played
- Morphological: Relating to the forms of words
 - Walk → Walking (suffix -ing changes the form and grammatical function)
 - Happy → Unhappy (prefix un- changes the meaning)

Two classes of words: Open vs. Closed

Closed class words

- ↓ Relatively fixed membership, meaning new words in this class are rarely coined
- ↓ **Function words:** Short, frequent words with grammatical function
 - Determiners: a, an, the
 - Pronouns: she, he, I
 - Prepositions: on, under, over, near, by,...
- ↓ PoS tags of such words are deterministic

Open class words

- ↓ Continually being created or borrowed
- ↓ Nouns (including proper nouns), verbs, adjectives, adverbs, & interjections

[Jurafsky & Martin Section 17.1](#) define many classes of words that you should familiarize yourself with: different types of nouns, adverbs, verbs, pronouns; particle, article, conjunction, complementizer, copula, modals, etc.

Fun facts about other languages 😊

- In **Korean**, the words corresponding to English adjectives act as a subclass of verbs, so what is in English an adjective “beautiful” acts in Korean like a verb meaning “to be beautiful”
- While many scholars believe that all human languages have the categories of noun and verb, others have argued that some languages, such as **Riau Indonesian** and **Tongan**, don’t even make this distinction

Ambiguity Resolution

I will book a room at the hotel

PRP MD VB DT NN IN DT NN

She is reading an interesting book

PRP VBZ VBG DT JJ NN

Number	Tag	Description
1.	CC	Coordinating conjunction
2.	CD	Cardinal number
3.	DT	Determiner
4.	EX	Existential <i>there</i>
5.	FW	Foreign word
6.	IN	Preposition or subordinating conjunction
7.	JJ	Adjective
8.	JJR	Adjective, comparative
9.	JJS	Adjective, superlative
10.	LS	List item marker
11.	MD	Modal
12.	NN	Noun, singular or mass
13.	NNS	Noun, plural
14.	NNP	Proper noun, singular
15.	NNPS	Proper noun, plural
16.	PDT	Predeterminer
17.	POS	Possessive ending
18.	PRP	Personal pronoun
19.	PRP\$	Possessive pronoun
20.	RB	Adverb
21.	RBR	Adverb, comparative
22.	RBS	Adverb, superlative
23.	RP	Particle
24.	SYM	Symbol
25.	TO	<i>to</i>
26.	UH	Interjection
27.	VB	Verb, base form
28.	VBD	Verb, past tense
29.	VBG	Verb, gerund or present participle
30.	VBN	Verb, past participle
31.	VBP	Verb, non-3rd person singular present
32.	VBZ	Verb, 3rd person singular present
33.	WDT	Wh-determiner
34.	WP	Wh-pronoun
35.	WP\$	Possessive wh-pronoun
36.	WRB	Wh-adverb

POS Tagging

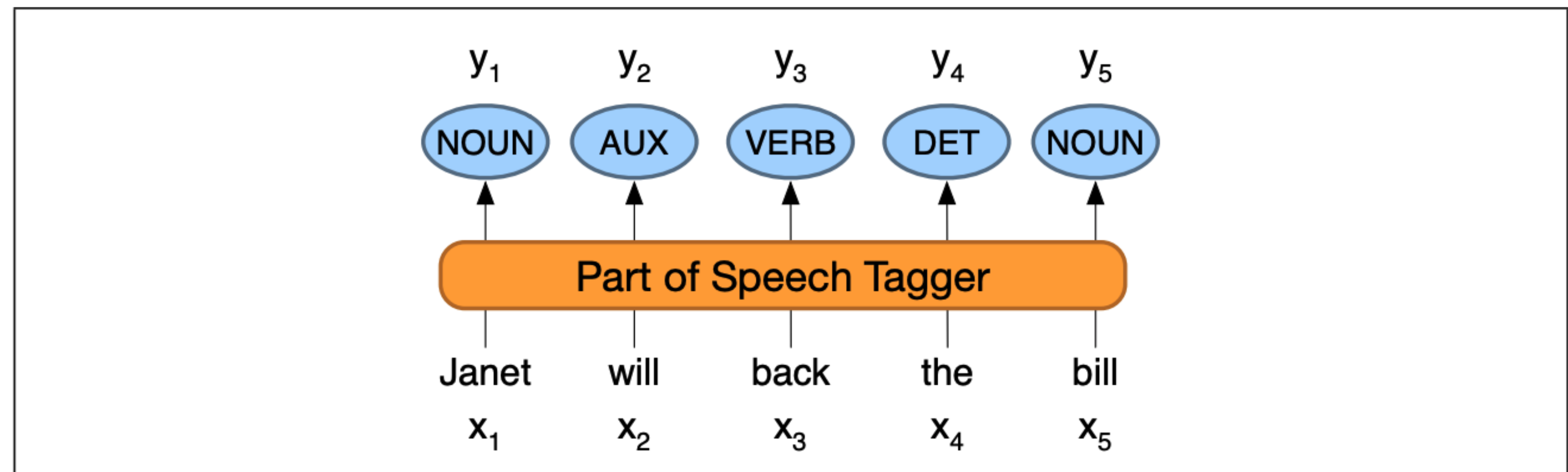
The task of assigning a part-of-speech to each word in a text

Input: A sequence $x_1, x_2, \dots, x_n$ of (tokenized) words and a **tagset**

Output: A sequence $y_1, y_2, \dots, y_n$ of tags, each output y_i corresponding exactly to one x_i

An example of **sequence labelling** or **sequence tagging**:

The task of assigning a label to each unit in a sequence, thus mapping a sequence of observations to a sequence of labels of the same length



Why POS tagging?

Can be useful for other NLP tasks

- ↓ Parsing: POS tagging can improve syntactic parsing [the next step in the classical NLP pipeline]
 - Whether a word is a noun or a verb tells us about likely neighboring words (nouns in English are preceded by determiners and adjectives, verbs by nouns)
 - It also tells us about syntactic structure (verbs have dependency links to nouns)
- ↓ MT: reordering of adjectives and nouns
 - Say from Spanish where adjectives come after the nouns to English where they precede the nouns
- ↓ Sentiment or affective tasks:
 - May want to distinguish adjectives or other POS
- ↓ Text-to-speech
 - How do we pronounce “lead” or “object”?

Useful for linguistic or language-analytic computational tasks

- ↓ Need to control for POS when studying linguistic change like creation of new words, or meaning shift
- ↓ Or control for POS in measuring meaning similarity or difference

How difficult is POS tagging in English?

Roughly 15% of word *types* are ambiguous

↓ Janet is always PROPN, hesitantly is always ADV

But those 15% tend to be very common,
so $\sim$ 60% of word *tokens* are ambiguous

E.g. type “*back*”

earnings growth took a back/ADJ seat

a small building in the back/NOUN

*a clear majority of senators **back**/VERB the bill*

*enable the country to buy **back**/PART debt*

I was twenty-one back/ADV then

Not all possible tags for a given word are equally likely!

Majority baseline (useful beyond PoS tagging):

Given a word, predict the PoS tag which is most frequent in the training corpus

The most-frequent-tag baseline has an accuracy of $\sim$ 92% on the sections 22-24 of the WSJ corpus, only 5% lower than the SOTA and human ceiling

Sources of information for POS tagging

Prior probabilities of word/tag

- “will” is usually an AUX

Identity of neighboring words

- “the” means the next word is probably not a verb

Morphology and wordshape:

- Prefixes
 - *unable*: un- $\Rightarrow$ ADJ
- Suffixes
 - *importantly*: -ly $\Rightarrow$ ADJ
- Capitalization
 - *Janet*: CAP $\Rightarrow$ PROP

Standard algorithms for POS tagging

Supervised Machine Learning:

- **Hidden Markov Models [today]**
- Conditional Random Fields (CRF) / Maximum Entropy Markov Models (MEMM)
- Neural sequence models (RNNs or Transformers)
- Pretrained (Large) Language Models (like BERT), finetuned

All required a hand-labeled training set, all about equal performance (97% on English)

All make use of information sources we discussed

- Via human created features: HMMs and CRFs
- Via representation learning: Neural LMs

Goals of Today's Lecture

Goal: Learn two classic sequence labeling tasks and a non-neural supervised approach to solving them

- Part-of-Speech (PoS) Tagging
- **Named Entity Recognition (NER)**
- Hidden Markov Model: Formulation
- Hidden Markov Model: Parameter estimation
- Viterbi algorithm

Named Entities (NEs)

Named entity typically means anything that can be referred to with a proper name

Most common 4 tags:

- ↓ PER (Person): “Marie Curie”
- ↓ LOC (Location): “New York City”
- ↓ ORG (Organization): “Stanford University”
- ↓ GPE (Geo-Political Entity): "Boulder, Colorado"

But the term is also extended to things that aren't entities, e.g., dates, times, prices

Entity classes can be specialized to a domain, e.g., GENE, PROTEIN, DISEASE, ...

They may range in specificity, e.g., ANIMAL, MAMMAL, DOG_BREED, ...

Often **multi-word phrases** ⇒ **Segmentation** problem: Where does the NE start/end?

Named Entity Recognition (NER)

The task of identify all spans in the text that denote some category of entities

As sequence labeling:

Input: A sequence $x_1, x_2, \dots, x_n$ of (tokenized) words and the **set of entity types**

Output: A sequence $y_1, y_2, \dots, y_n$ of **(B)IO(ES)** entity tags, each output y_i corresponding exactly to one x_i

(B)IO(ES) Tagging

- B: token that *begins* a span
- I: tokens *inside* a span
- O: tokens outside of any span
- E: token that *ends* a span
- S: singleton

Words	IO Label	BIO Label	BIOES Label
Jane	I-PER	B-PER	B-PER
Villanueva	I-PER	I-PER	E-PER
of	O	O	O
United	I-ORG	B-ORG	B-ORG
Airlines	I-ORG	I-ORG	I-ORG
Holding	I-ORG	I-ORG	E-ORG
discussed	O	O	O
the	O	O	O
Chicago	I-LOC	B-LOC	S-LOC
route	O	O	O
.	O	O	O

Ambiguity Resolution

Type	Tag	Sample Categories	Example sentences
People	PER	people, characters	Turing is a giant of computer science.
Organization	ORG	companies, sports teams	The IPCC warned about the cyclone.
Location	LOC	regions, mountains, seas	Mt. Sanitas is in Sunshine Canyon .
Geo-Political Entity	GPE	countries, states	Palo Alto is raising the fees for parking.

Washington was born into slavery on the farm of James Burroughs. PER

Washington went up 2 games to 1 in the four-game series. ORG

Blair arrived in **Washington** for what may well be his last state visit. LOC

In June, **Washington** passed a primary seatbelt law. GPE

Why NER?

Can be useful for other NLP tasks

- ↳ Targeted aspect-based sentiment analysis: Consumer's sentiment toward a particular company or person?
- ↳ Question Answering: Answer questions about an entity?
- ↳ Information Extraction: Extracting facts about entities from text

Or linguistic or language-analytic computational tasks

- ↳ For example, you may want to study how British novels depicted India during the colonial period
- ↳ Using a corpus of novels, you could use NER to:
 - Extract **locations** like Calcutta, Delhi, and Himalayas to track representations of geography
 - Identify **people** such as Rama or Lord Cornwallis to study how individuals were characterized
 - Detect **dates/events** like 1857 (Indian Rebellion) for cultural context

Standard algorithms for NER

Supervised Machine Learning given a human-labeled training set of text annotated with tags

- **Hidden Markov Models [today]**
- Conditional Random Fields (CRF) / Maximum Entropy Markov Models (MEMM)
- Neural sequence models (RNNs or Transformers)
- Pretrained (Large) Language Models (like BERT), finetuned

Goals of Today's Lecture

Goal: Learn two classic sequence labeling tasks and a non-neural supervised approach to solving them

- Part-of-Speech (PoS) Tagging
- Named Entity Recognition (NER)
- **Hidden Markov Model: Formulation**
- Hidden Markov Model: Parameter estimation
- Viterbi algorithm

Hidden Markov Model (HMM) is a probabilistic sequence model: given a sequence of units (words, letters, morphemes, sentences, whatever), it computes a probability distribution over possible sequences of labels and chooses the best label sequence.

Can you recall when we mentioned Markov before?

When did we last calculate a probability distribution for some potential sequences?

Components of a (Discrete) Markov chain

$Q = q_1 q_2 \dots q_N$... a set of N **states=observations** [random variables which can take on values from some set]

Components of a (Discrete) Markov chain (cont.)

$Q = q_1 q_2 \dots q_N$... a set of N **states=observations** [random variables which can take on values from some set]

$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{N1} \\ \vdots & \vdots & \dots & \vdots \\ a_{N1} & a_{N2} & \dots & a_{NN} \end{bmatrix}$... a **transition probability** matrix A, each a_{ij} representing the probability of moving from state i to state j , s.t. $\sum_{j=1}^N a_{ij} = 1 \quad \forall i$

Components of a (Discrete) Markov chain (cont.)

$Q = q_1 q_2 \dots q_N$... a set of N **states=observations** [random variables which can take on values from some set]

$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{N1} \\ \vdots & \vdots & \dots & \vdots \\ a_{N1} & a_{N2} & \dots & a_{NN} \end{bmatrix}$... a **transition probability** matrix A, each a_{ij} representing the probability of moving from state i to state j , s.t. $\sum_{j=1}^N a_{ij} = 1 \quad \forall i$

$\pi = \pi_1, \pi_2, \dots, \pi_N$... an **initial probability distribution** over states

π_i ... is the probability that the Markov chain will start in state i $\sum_{i=1}^N \pi_i = 1$

Some states j may have $\pi_j = 0$, meaning that they cannot be initial states

Components of a (Discrete) Markov chain (cont.)

$Q = q_1 q_2 \dots q_N$... a set of N **states=observations** [random variables which can take on values from some set]

$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{N1} \\ \vdots & \vdots & \dots & \vdots \\ a_{N1} & a_{N2} & \dots & a_{NN} \end{bmatrix}$... a **transition probability** matrix A, each a_{ij} representing the probability of moving from state i to state j , s.t. $\sum_{j=1}^N a_{ij} = 1 \quad \forall i$

$\pi = \pi_1, \pi_2, \dots, \pi_N$... an **initial probability distribution** over states

π_i ... is the probability that the Markov chain will start in state i $\sum_{i=1}^N \pi_i = 1$

Some states j may have $\pi_j = 0$, meaning that they cannot be initial states

(1st order) Markov assumption:

$$\mathbb{P}(q_i = x | q_1 \dots q_{i-1}) = \mathbb{P}(q_i = x | q_{i-1})$$

Components of a Hidden Markov Model

Observable vs. **hidden** events: the former we observe directly, the latter must be inferred

- We see words, and must infer the part-of-speech tags from the word sequence

We build HMM from the same components as the Markov chain and introduce:

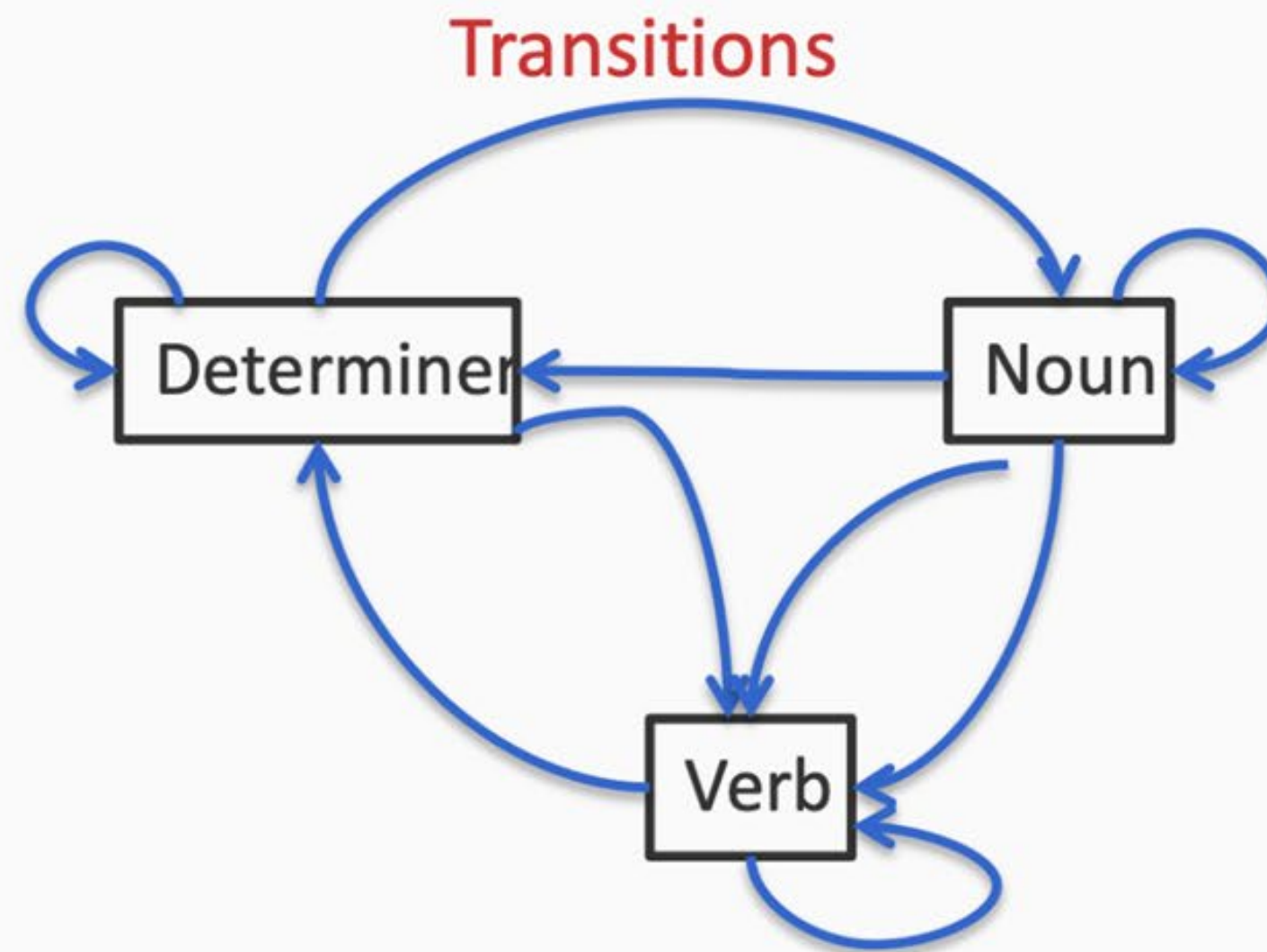
$$B = \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1V} \\ \vdots & \vdots & \cdots & \vdots \\ b_{N1} & b_{N2} & \cdots & b_{NV} \end{bmatrix} \quad \dots \text{ a matrix of **emission probabilities**, each expressing the probability of an output observation } O_t \text{ (drawn from a vocabulary } V = v_1, v_2, \dots, v_V \text{) being generated from a hidden state } q_i$$

And we make another assumption besides the Markov assumption:

- **Output independence:** The probability of an output observation O_i depends only on the state that produced the observation q_i and not on any other states or any other observation

$$\mathbb{P}(O_i | q_1, \dots, q_T, O_1, \dots, O_T) = \mathbb{P}(O_i | q_i)$$

Toy part of speech example



*Each edge here is associated with a **transition probability***

Initial

$P(\text{Determiner}) = 0.9$
 $P(\text{Noun}) = 0.08$
 $P(\text{Verb}) = 0.02$

***Initial probabilities:** What is the probability that the sequence starts in a certain state?*

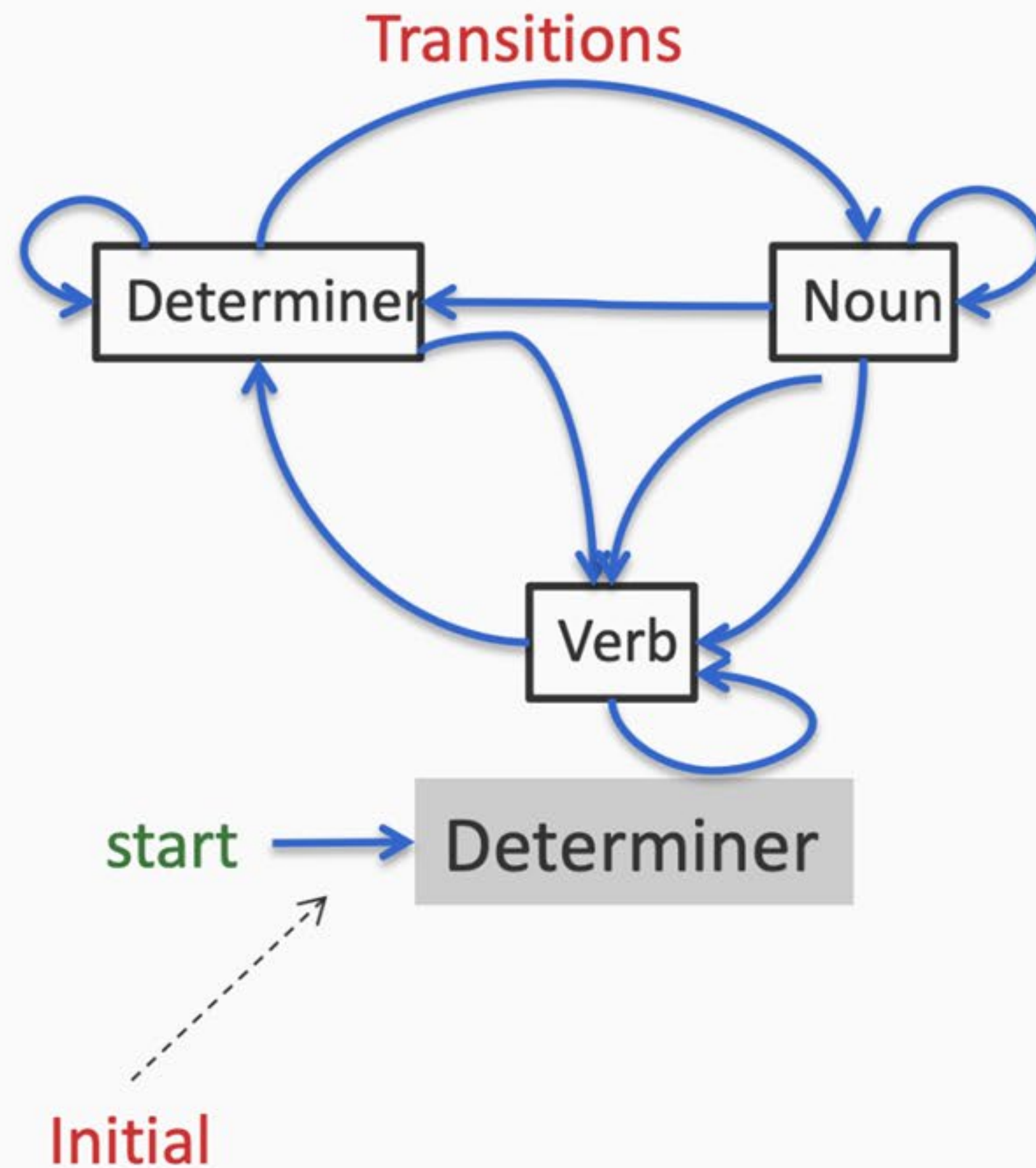
Emissions

$P(\text{The} \mid \text{Determiner}) = 0.5$
 $P(\text{A} \mid \text{Determiner}) = 0.3$
 $P(\text{An} \mid \text{Determiner}) = 0.1$
 $P(\text{Fed} \mid \text{Determiner}) = 0$
...

$P(\text{Fed} \mid \text{Noun}) = 0.001$
 $P(\text{raises} \mid \text{Noun}) = 0.04$
 $P(\text{interest} \mid \text{Noun}) = 0.07$
 $P(\text{The} \mid \text{Noun}) = 0$
...

***Emission probabilities:** Given that the system is in a certain state, these are probabilities that it will emit a certain observation*

Toy part of speech example

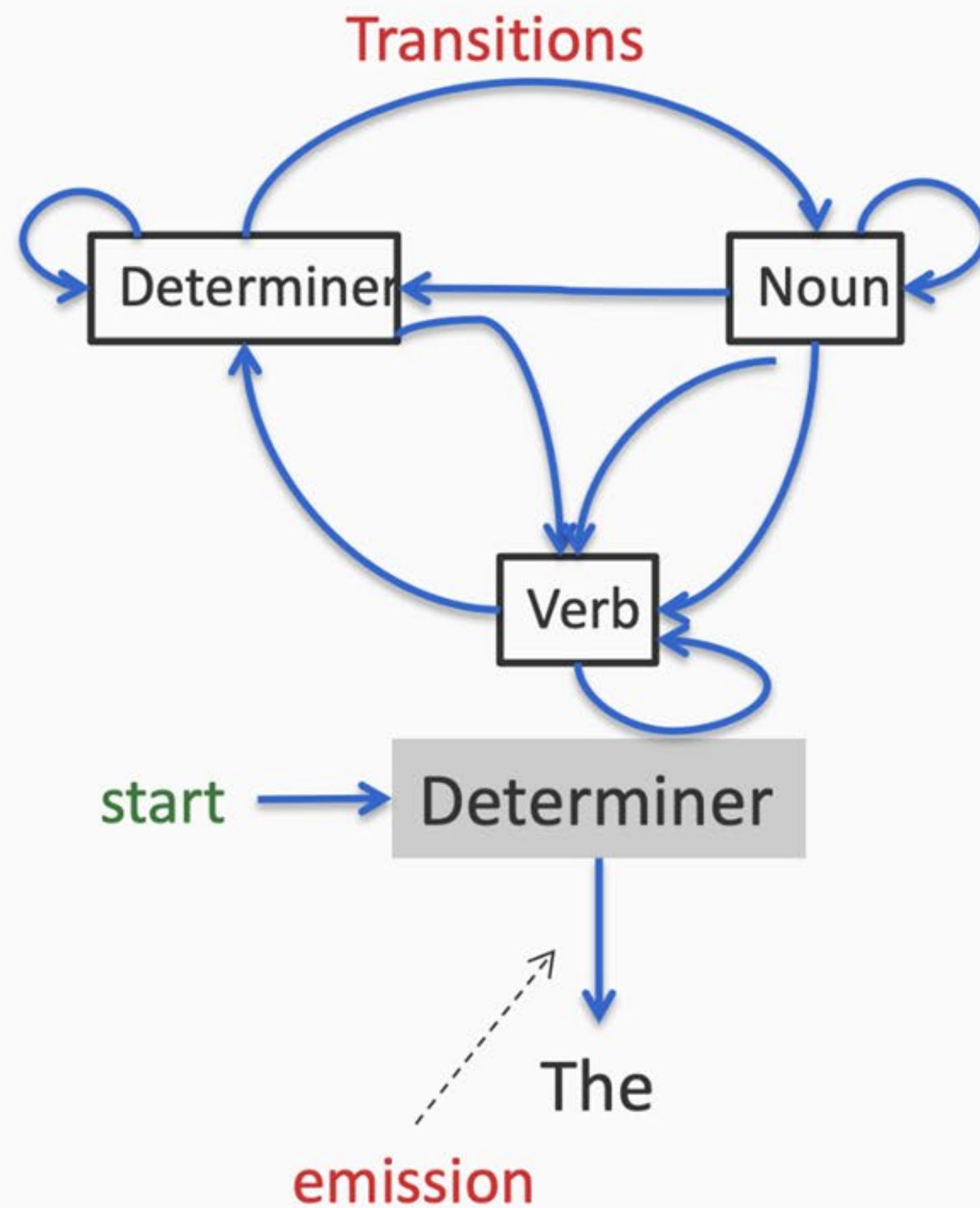


$P(\text{The} \mid \text{Determiner}) = 0.5$
 $P(\text{A} \mid \text{Determiner}) = 0.3$
 $P(\text{An} \mid \text{Determiner}) = 0.1$
 $P(\text{Fed} \mid \text{Determiner}) = 0$
...

Emissions

$P(\text{Fed} \mid \text{Noun}) = 0.001$
 $P(\text{raises} \mid \text{Noun}) = 0.04$
 $P(\text{interest} \mid \text{Noun}) = 0.07$
 $P(\text{The} \mid \text{Noun}) = 0$
...

Toy part of speech example



Emissions

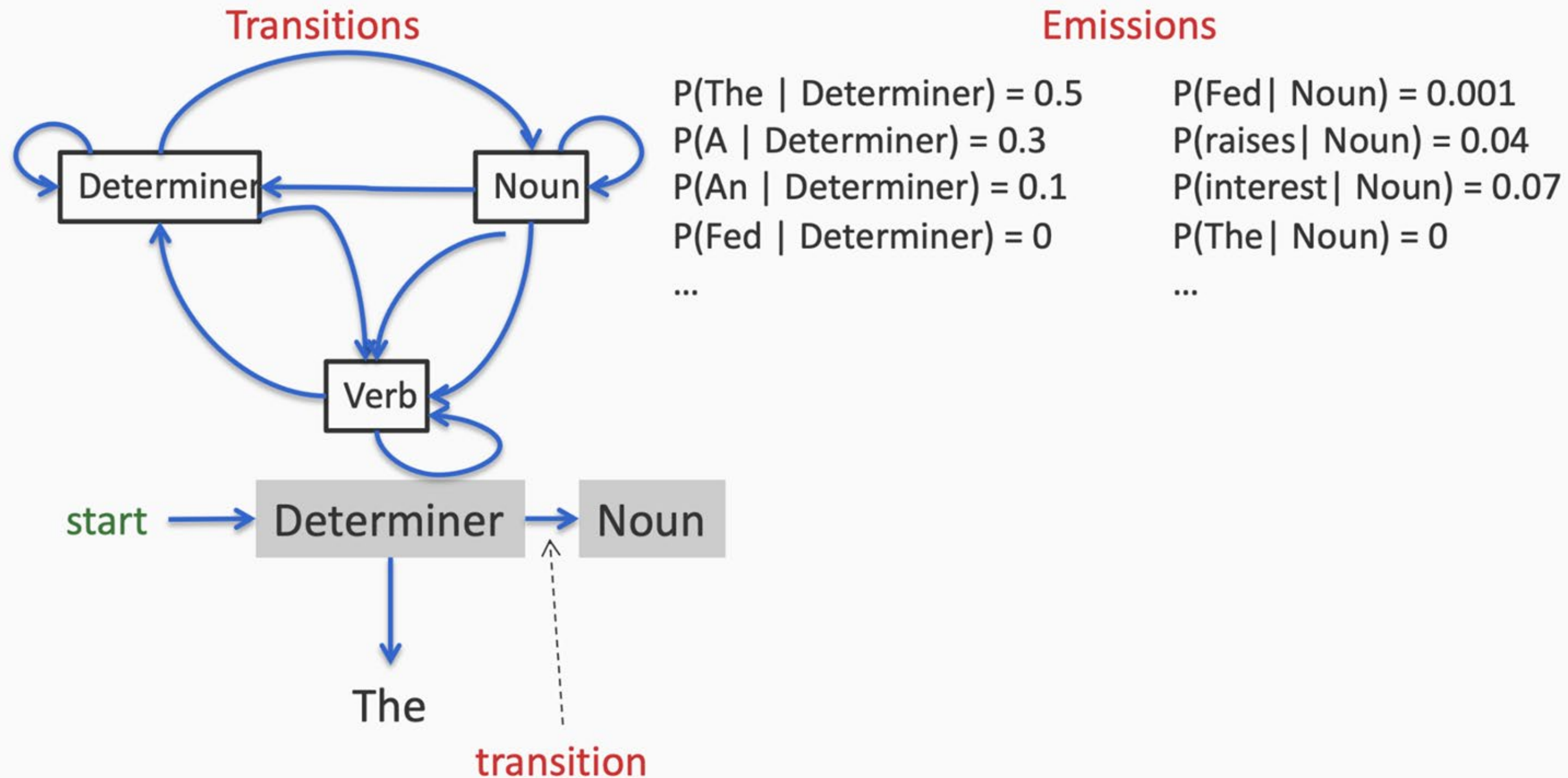
$P(\text{The} \mid \text{Determiner}) = 0.5$
 $P(\text{A} \mid \text{Determiner}) = 0.3$
 $P(\text{An} \mid \text{Determiner}) = 0.1$
 $P(\text{Fed} \mid \text{Determiner}) = 0$
...

$P(\text{Fed} \mid \text{Noun}) = 0.001$
 $P(\text{raises} \mid \text{Noun}) = 0.04$
 $P(\text{interest} \mid \text{Noun}) = 0.07$
 $P(\text{The} \mid \text{Noun}) = 0$
...

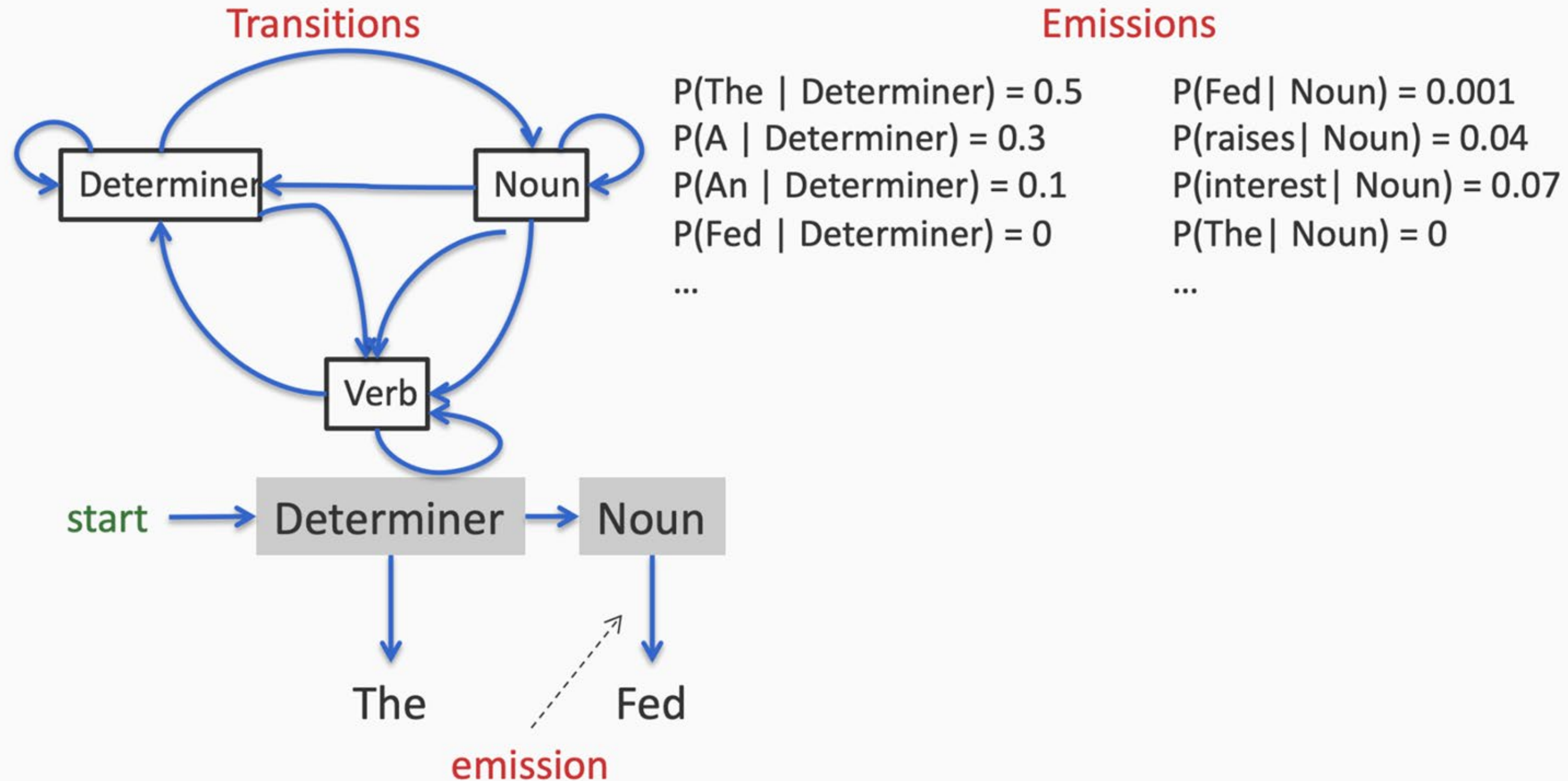
emission

The

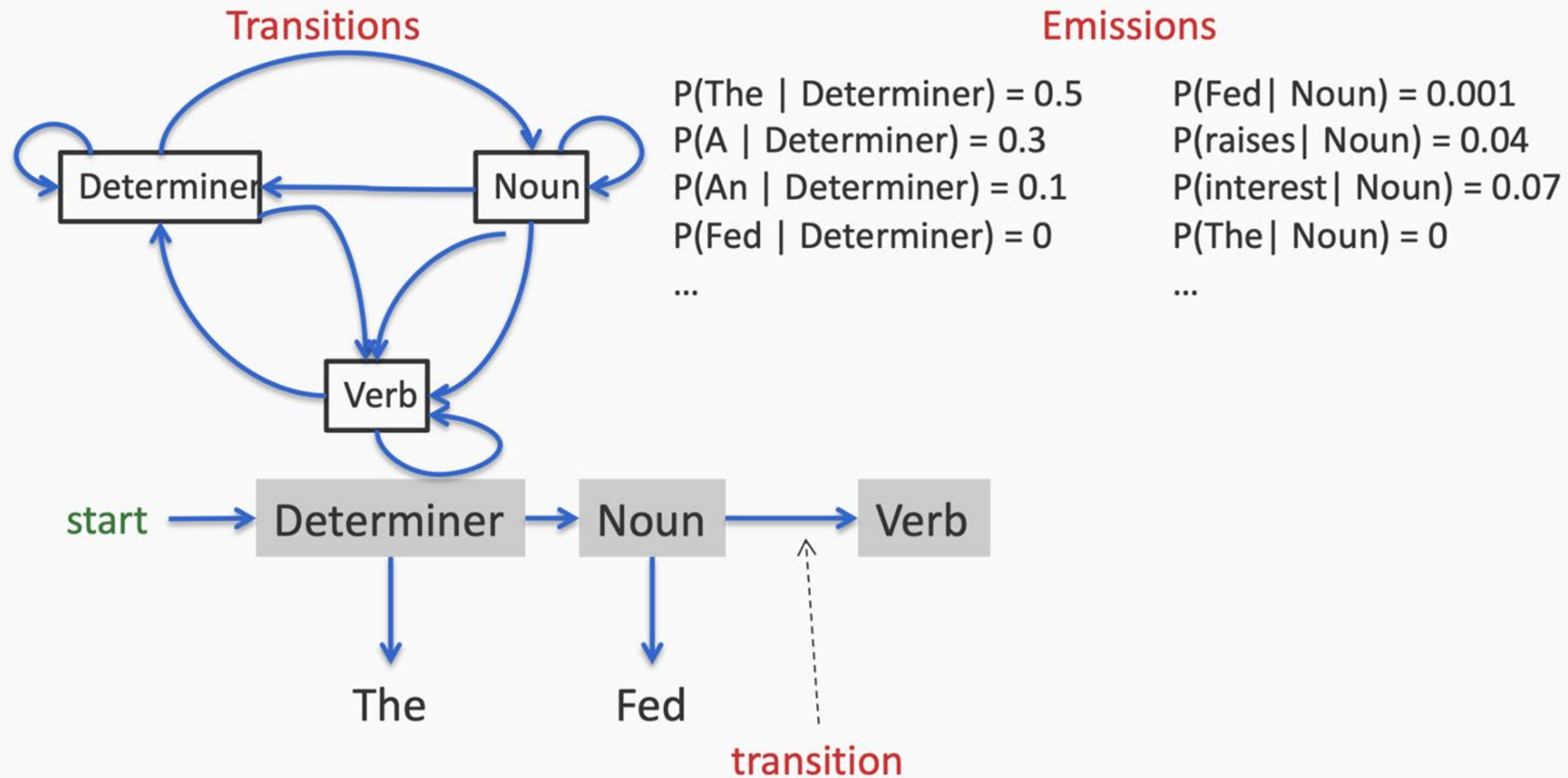
Toy part of speech example



Toy part of speech example



Toy part of speech example



HMM Tagger

$w_1, w_2, \dots, w_N$... a sequence of observed words

$$\hat{t}_{1:N} = \operatorname{argmax}_{t_{1:N}} \mathbb{P}(t_1, \dots, t_N | w_1, \dots, w_N)$$

Bayes rule to turn to the process we've just seen with the toy example

$$= \operatorname{argmax}_{t_{1:N}} \frac{\mathbb{P}(w_1, \dots, w_N | t_1, \dots, t_N) \mathbb{P}(t_1, \dots, t_N)}{\mathbb{P}(w_1, \dots, w_N)}$$

The denominator independent of tags

$$= \operatorname{argmax}_{t_{1:N}} \mathbb{P}(w_1, \dots, w_N | t_1, \dots, t_N) \mathbb{P}(t_1, \dots, t_N)$$

Markov and output independence assumptions

$$\approx \operatorname{argmax}_{t_{1:N}} \prod_{i=1}^N \mathbb{P}(w_i | t_i) \cdot \mathbb{P}(t_1) \prod_{i=2}^N \mathbb{P}(t_i | t_{i-1})$$

Let's use the notation we introduced

$$\approx \operatorname{argmax}_{t_{1:N}} \prod_{i=1}^N B_{t_i, w_i} \cdot \pi_{t_1} \prod_{i=2}^N A_{t_{i-1}, t_i}$$

Once we have estimated emission, initial, and transition probabilities, all we need to do to get the most probable sequence of tags for a given sequence of words is to plug the probabilities into this equation for every possible sequence of tags and return the sequence that maximizes the equation's value

How to calculate π , A , B from observations?

Two possible scenarios:

1. We are given a dataset of sequences labeled with states...
...and we have to learn the parameters of the HMM

- * **Supervised learning**

2. We are given only a collection of sequences...
...and we have to learn the parameters of the HMM

- * **Unsupervised learning**

- * **Baum–Welch algorithm: a special case of the expectation–maximization (EM) algorithm used to find the unknown parameters of a hidden Markov model (HMM)**

Supervised learning of HMM

The maximum likelihood principle

a sequence of words

$$\mathcal{D} = (\mathbf{w}^{(k)}, \mathbf{t}^{(k)})_{k=1}^D$$

a sequence of tags

$$\begin{aligned} & \operatorname{argmax}_{\pi, A, B} \prod_{k=1}^D \mathbb{P}(\mathbf{w}^{(k)}, \mathbf{t}^{(k)}) \\ & \approx \operatorname{argmax}_{\pi, A, B} \prod_{k=1}^D \prod_{i=1}^N B_{t_i^{(k)}, w_i^{(k)}} \cdot \pi_{t_1^{(k)}} \prod_{i=2}^N A_{t_{i-1}^{(k)}, t_i^{(k)}} \end{aligned}$$

π, A, B can be estimated separately by counting using a *tagged* training corpus

Supervised learning of HMM

$$\mathcal{D} = (\mathbf{w}^{(k)}, \mathbf{t}^{(k)})_{k=1}^D$$

a special start token

$$\pi_t = \frac{\text{count}(\text{start} \rightarrow t)}{|\mathcal{D}|}$$

$$A_{t',t} = \frac{\text{count}(t \rightarrow t')}{\text{count}(t)}$$

$$B_{t,w} = \frac{\text{count}(t \rightarrow w)}{\text{count}(t)}$$

for every possible tag $\mathbf{t}$ in the tagset and word $\mathbf{w}$ in the vocabulary

Add small constants to the counts to avoid zero probabilities (smoothing)

How many possible sequences?

The

Fed

raises

interest

rates

Suppose each word allows only the following tags

Determiner

Verb

Verb

Verb

Verb

Noun

Noun

Noun

Noun

1

2

2

2

2

How many possible sequences?

The

Fed

raises

interest

rates

Suppose each word allows only the following tags

Determiner

Verb

Verb

Verb

Verb

Noun

Noun

Noun

Noun

1

2

2

2

2

In this simple case, $1 \times 2 \times 2 \times 2 \times 2 = 16$ possible sequences exist

Given an observed sequence, and a model (π, A, B) how to efficiently calculate the most probable state sequence?

$t_1, t_2, \dots, t_N$... a sequence of tags

$w_1, w_2, \dots, w_N$... a sequence of observed words

$$\hat{t}_{1:N} = \operatorname{argmax}_{t_{1:N}} \mathbb{P}(t_1, \dots, t_N | w_1, \dots, w_N) \approx \operatorname{argmax}_{t_{1:N}} \prod_{i=1}^N B_{t_i, w_i} \cdot \pi_{t_1} \prod_{i=2}^N A_{t_{i-1}, t_i}$$

Naïve approaches

1. Try out every sequence and return the highest scoring one
 - Correct, but slow, $O(\text{num-possible-states}^{\text{sequence-length}})$
2. Greedy search
 - The best tag given the previously chosen tag and observed word
 - Incorrect, but fast $O(\text{sequence-length}) = \arg \max_t \mathbb{P}(t | t_{i-1}) P(w_i | t)$

Viterbi algorithm: $O(\text{sequence-length} \times \text{num-possible-states}^2)$

Solution: Use the independence assumptions

$t_1, t_2, \dots, t_N$... a sequence of tags

$w_1, w_2, \dots, w_N$... a sequence of observed words

$$\hat{t}_{1:N} = \operatorname{argmax}_{t_{1:N}} \mathbb{P}(t_1, \dots, t_N | w_1, \dots, w_N) \approx \operatorname{argmax}_{t_{1:N}} \prod_{i=1}^N B_{t_i, w_i} \cdot \pi_{t_1} \prod_{i=2}^N A_{t_{i-1}, t_i}$$

Take advantage of the first order Markov assumption

The state for any observation is only influenced by the previous state, the next state and the observation itself

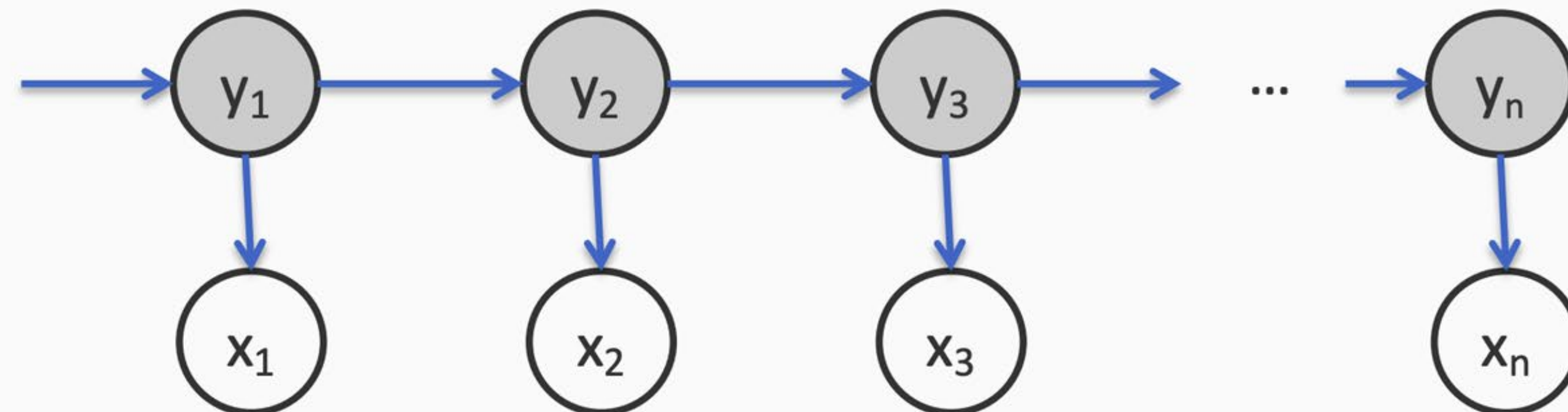
Given the adjacent labels, the others do not matter

Suggests a recursive algorithm

Deriving the recursive algorithm

$$P(x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n) = P(y_1) \prod_{i=1}^{n-1} P(y_{i+1}|y_i) \prod_{i=1}^n P(x_i|y_i)$$

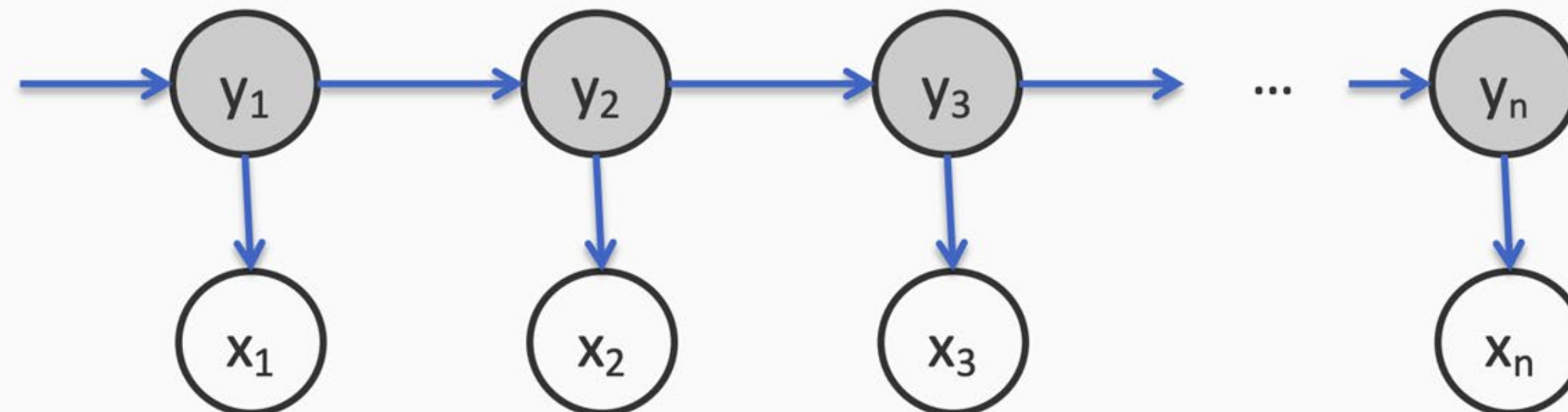
What we want: An assignment to all the y_i 's that maximizes this product



Deriving the recursive algorithm

$$P(x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n) = P(y_1) \prod_{i=1}^{n-1} P(y_{i+1}|y_i) \prod_{i=1}^n P(x_i|y_i)$$

$$\max_{y_1, y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots P(y_2|y_1)P(x_2|y_2)P(y_1)P(x_1|y_1)$$

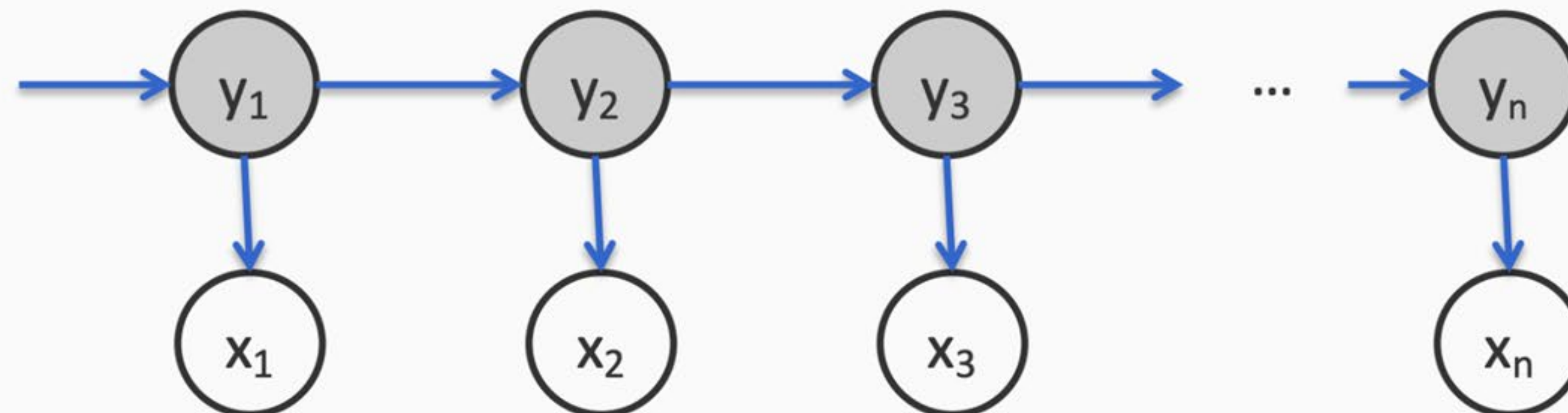


Deriving the recursive algorithm

$$P(x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n) = P(y_1) \prod_{i=1}^{n-1} P(y_{i+1}|y_i) \prod_{i=1}^n P(x_i|y_i)$$

$$\max_{y_1, y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots P(y_2|y_1)P(x_2|y_2) \boxed{P(y_1)}P(x_1|y_1)$$

Initial probability

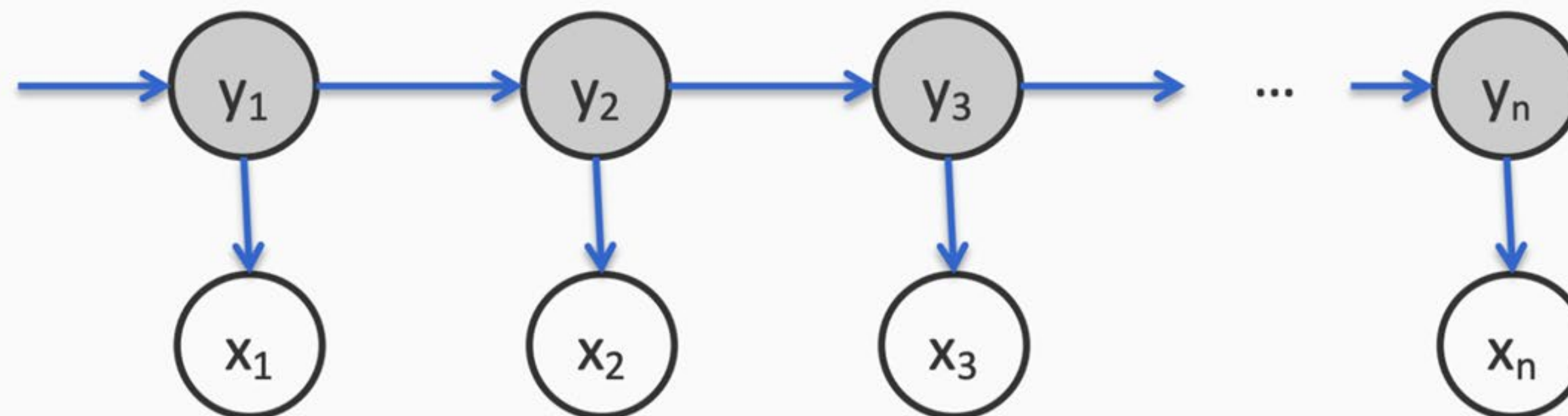


Deriving the recursive algorithm

$$P(x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n) = P(y_1) \prod_{i=1}^{n-1} P(y_{i+1}|y_i) \prod_{i=1}^n P(x_i|y_i)$$

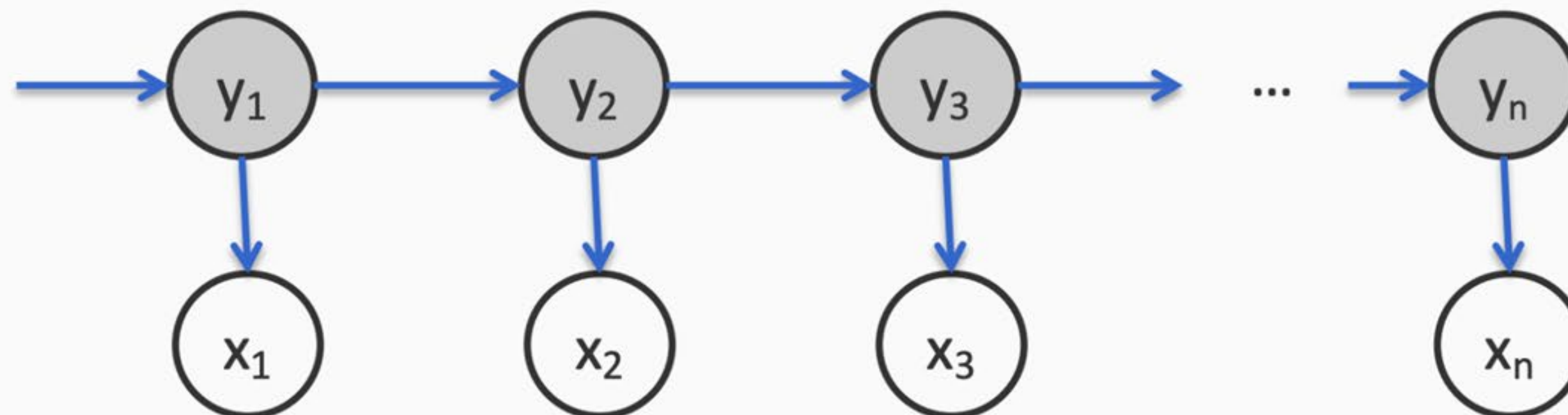
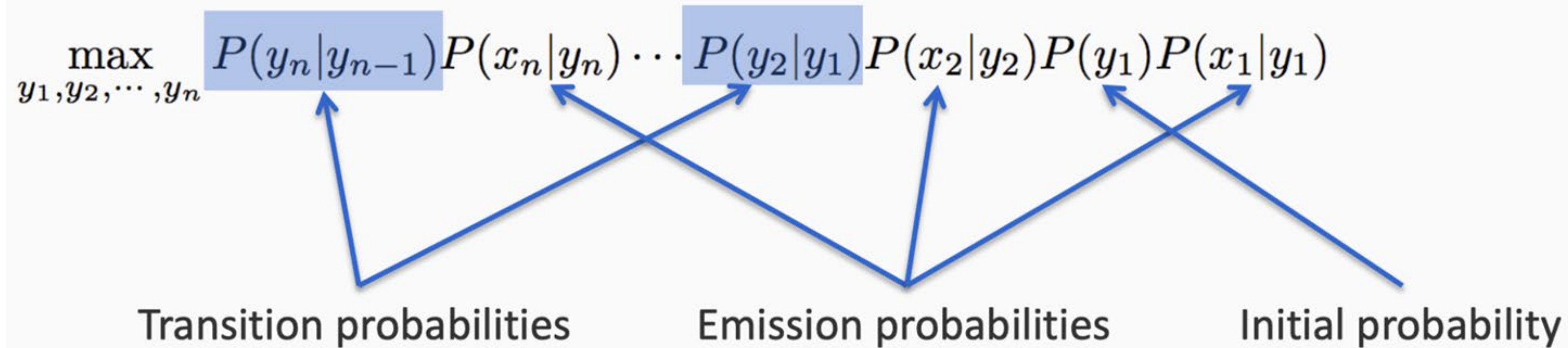
$$\max_{y_1, y_2, \dots, y_n} P(y_n|y_{n-1}) \underbrace{P(x_n|y_n)} \cdots P(y_2|y_1) \underbrace{P(x_2|y_2)} P(y_1) \underbrace{P(x_1|y_1)}$$

Emission probabilities Initial probability



Deriving the recursive algorithm

$$P(x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n) = P(y_1) \prod_{i=1}^{n-1} P(y_{i+1}|y_i) \prod_{i=1}^n P(x_i|y_i)$$



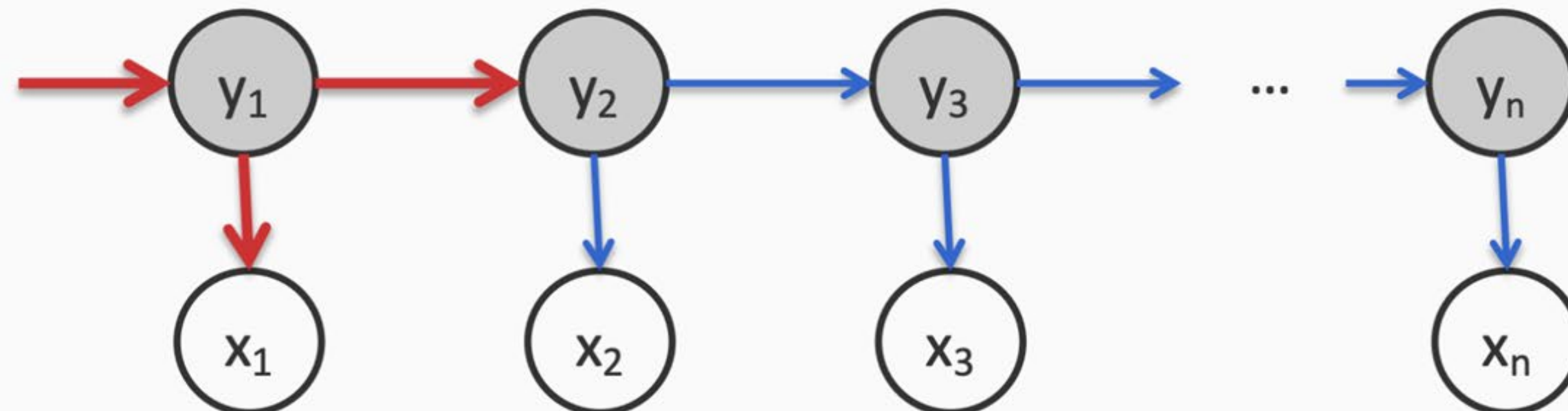
Deriving the recursive algorithm

$$P(x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n) = P(y_1) \prod_{i=1}^{n-1} P(y_{i+1}|y_i) \prod_{i=1}^n P(x_i|y_i)$$

$$\begin{aligned} & \max_{y_1, y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots P(y_2|y_1)P(x_2|y_2)P(y_1)P(x_1|y_1) \\ &= \max_{y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots \max_{y_1} P(y_2|y_1)P(x_2|y_2)P(y_1)P(x_1|y_1) \end{aligned}$$

Only a few factors depend on y_1 so we rearrange the product such that we place all those factors to the right

We can move $\max_{y_1}$ to the right too because other terms do not depend on y_1



Deriving the recursive algorithm

$$P(x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n) = P(y_1) \prod_{i=1}^{n-1} P(y_{i+1}|y_i) \prod_{i=1}^n P(x_i|y_i)$$

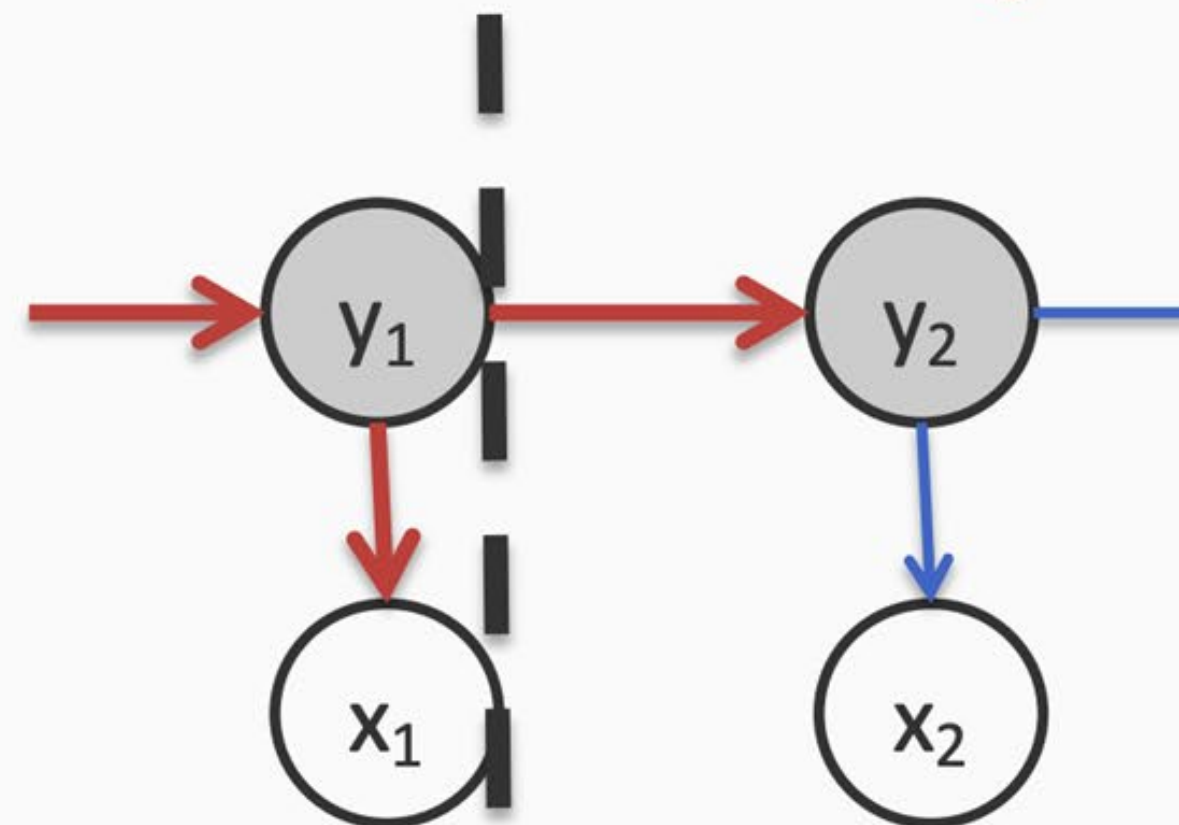
$$\max_{y_1, y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots P(y_2|y_1)P(x_2|y_2)P(y_1)P(x_1|y_1)$$

$$= \max_{y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots \max_{y_1} P(y_2|y_1)P(x_2|y_2)P(y_1)P(x_1|y_1)$$

$$= \max_{y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots \max_{y_1} P(y_2|y_1)P(x_2|y_2) \text{score}_1(y_1)$$

Abstract away the score for all decisions till here into **score₁**

$$\text{score}_1(s) = P(s)P(x_1|s)$$



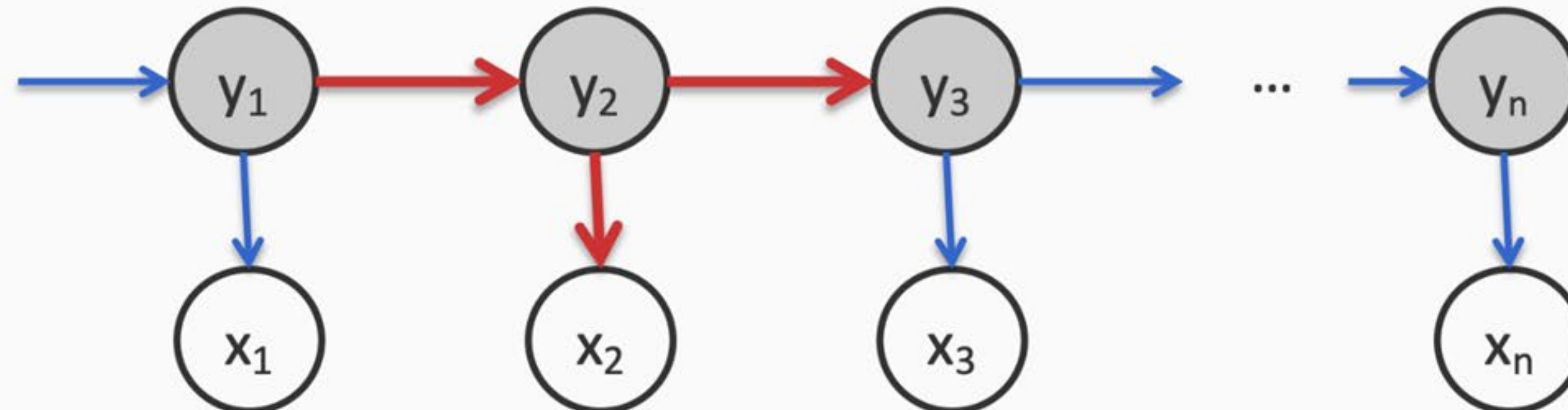
Abstract away the last two terms into something that we will give a special name **score₁**

s is a symbol for any state

Deriving the recursive algorithm

$$P(x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n) = P(y_1) \prod_{i=1}^{n-1} P(y_{i+1}|y_i) \prod_{i=1}^n P(x_i|y_i)$$

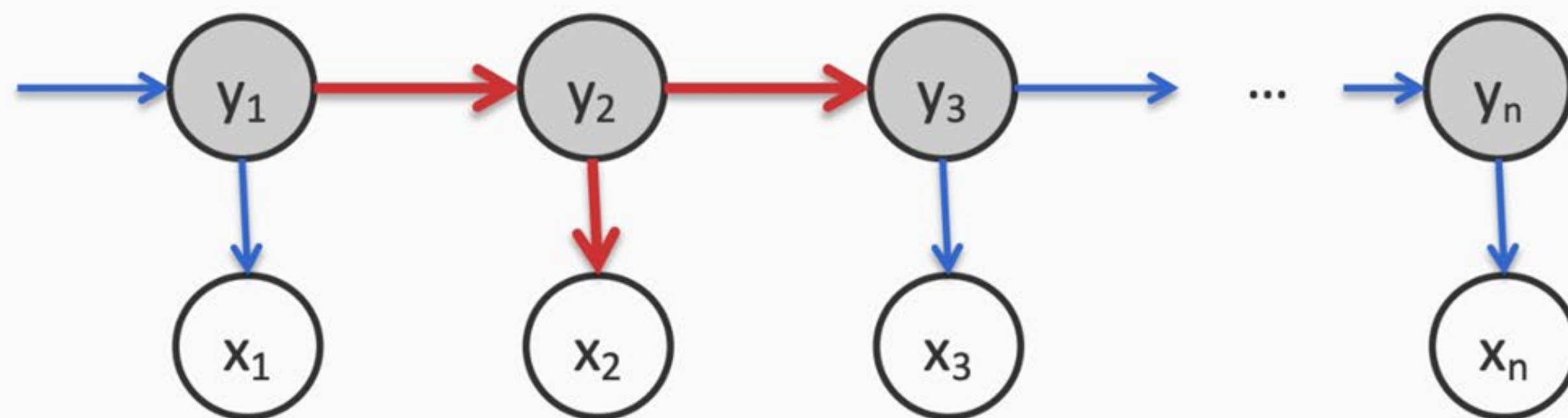
$$\begin{aligned} & \max_{y_1, y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots P(y_2|y_1)P(x_2|y_2)P(y_1)P(x_1|y_1) \\ &= \max_{y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots \max_{y_1} P(y_2|y_1)P(x_2|y_2)P(y_1)P(x_1|y_1) \\ &= \max_{y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots \max_{y_1} P(y_2|y_1)P(x_2|y_2) \text{score}_1(y_1) \\ &= \max_{y_3, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots \max_{y_2} P(y_3|y_2)P(x_3|y_3) \max_{y_1} P(y_2|y_1)P(x_2|y_2) \text{score}_1(y_1) \end{aligned}$$



Deriving the recursive algorithm

$$P(x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n) = P(y_1) \prod_{i=1}^{n-1} P(y_{i+1}|y_i) \prod_{i=1}^n P(x_i|y_i)$$

$$\begin{aligned} & \max_{y_1, y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots P(y_2|y_1)P(x_2|y_2)P(y_1)P(x_1|y_1) \\ &= \max_{y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots \max_{y_1} P(y_2|y_1)P(x_2|y_2)P(y_1)P(x_1|y_1) \\ &= \max_{y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots \max_{y_1} P(y_2|y_1)P(x_2|y_2) \text{score}_1(y_1) \\ &= \max_{y_3, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots \max_{y_2} P(y_3|y_2)P(x_3|y_3) \max_{y_1} P(y_2|y_1)P(x_2|y_2) \text{score}_1(y_1) \end{aligned}$$



Only terms that depend on y_2

Deriving the recursive algorithm

$$P(x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n) = P(y_1) \prod_{i=1}^{n-1} P(y_{i+1}|y_i) \prod_{i=1}^n P(x_i|y_i)$$

$$\max_{y_1, y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots P(y_2|y_1)P(x_2|y_2)P(y_1)P(x_1|y_1)$$

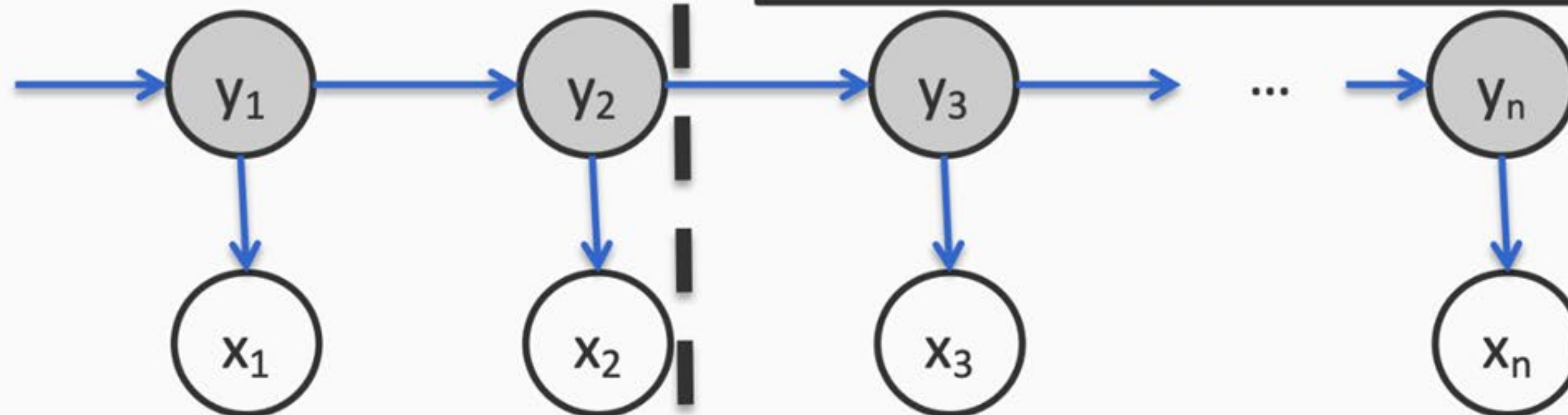
$$= \max_{y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots \max_{y_1} P(y_2|y_1)P(x_2|y_2)P(y_1)P(x_1|y_1)$$

$$= \max_{y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots \max_{y_1} P(y_2|y_1)P(x_2|y_2) \text{score}_1(y_1)$$

$$= \max_{y_3, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots \max_{y_2} P(y_3|y_2)P(x_3|y_3) \max_{y_1} P(y_2|y_1)P(x_2|y_2) \text{score}_1(y_1)$$

$$= \max_{y_3, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots \max_{y_2} P(y_3|y_2)P(x_3|y_3) \text{score}_2(y_2)$$

$$\text{score}_i(s) = \max_{y_{i-1}} P(s|y_{i-1})P(x_i|s) \text{score}_{i-1}(y_{i-1})$$



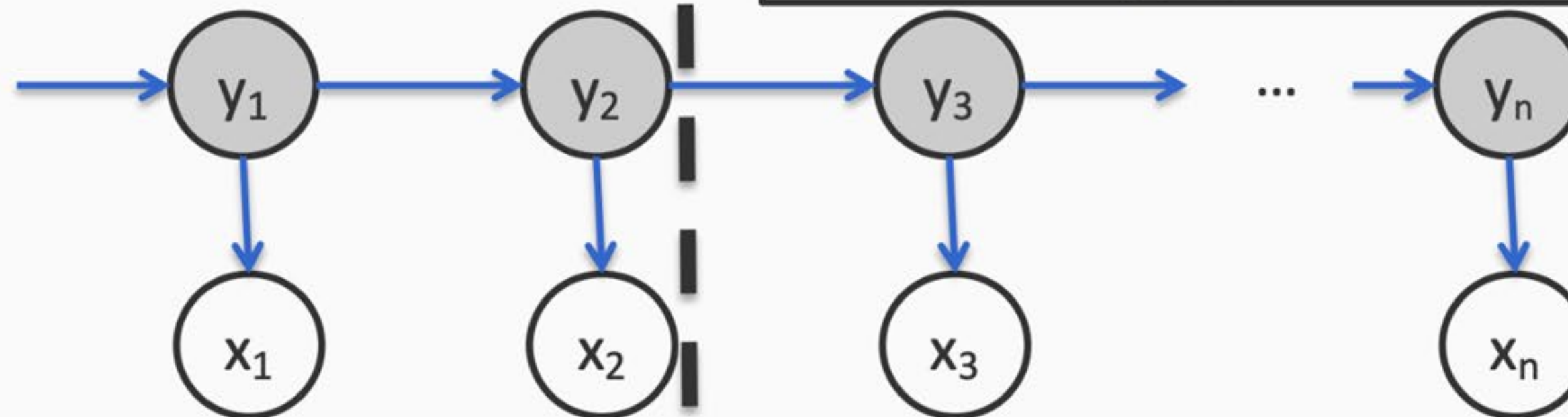
Abstract away the score for all decisions till here into **score**

Deriving the recursive algorithm

$$P(x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n) = P(y_1) \prod_{i=1}^{n-1} P(y_{i+1}|y_i) \prod_{i=1}^n P(x_i|y_i)$$

$$\begin{aligned} & \max_{y_1, y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots P(y_2|y_1)P(x_2|y_2)P(y_1)P(x_1|y_1) \\ &= \max_{y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots \max_{y_1} P(y_2|y_1)P(x_2|y_2)P(y_1)P(x_1|y_1) \\ &= \max_{y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots \max_{y_1} P(y_2|y_1)P(x_2|y_2) \text{score}_1(y_1) \\ &= \max_{y_3, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots \max_{y_2} P(y_3|y_2)P(x_3|y_3) \max_{y_1} P(y_2|y_1)P(x_2|y_2) \text{score}_1(y_1) \\ &= \max_{y_3, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots \max_{y_2} P(y_3|y_2)P(x_3|y_3) \text{score}_2(y_2) \end{aligned}$$

$$\text{score}_i(s) = \max_{y_{i-1}} P(s|y_{i-1})P(x_i|s) \text{score}_{i-1}(y_{i-1})$$



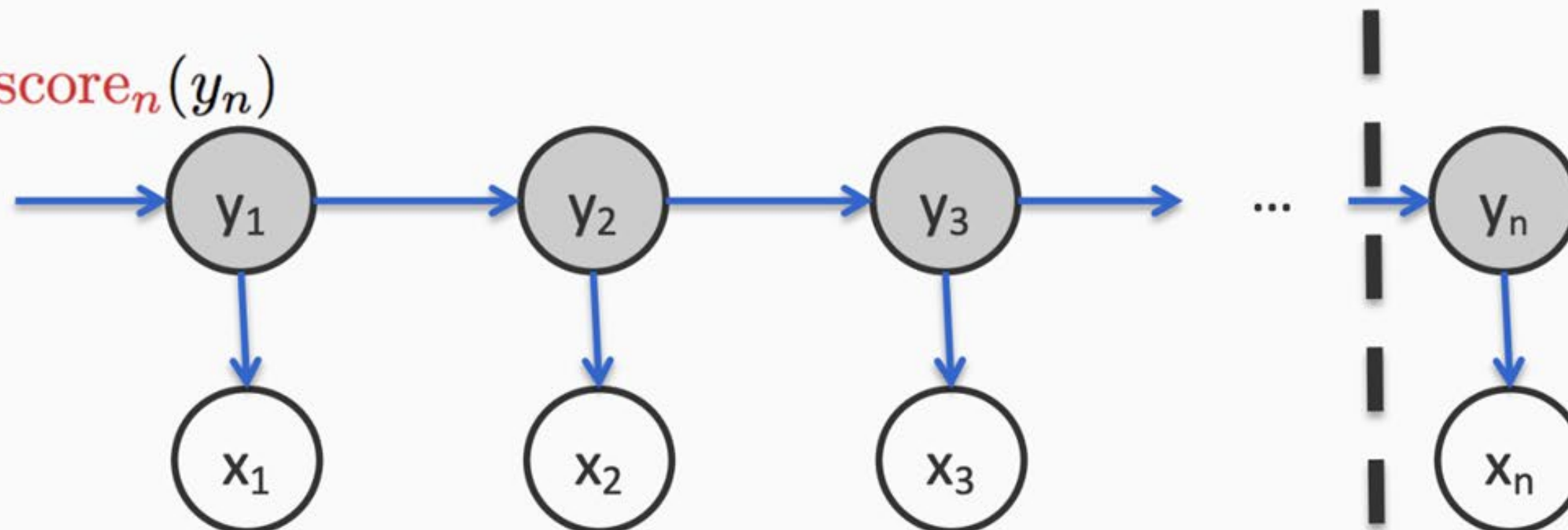
Abstract away the score for all decisions till here into **score**

Deriving the recursive algorithm

$$P(x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n) = P(y_1) \prod_{i=1}^{n-1} P(y_{i+1}|y_i) \prod_{i=1}^n P(x_i|y_i)$$

$$\begin{aligned} & \max_{y_1, y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots P(y_2|y_1)P(x_2|y_2)P(y_1)P(x_1|y_1) \\ &= \max_{y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots \max_{y_1} P(y_2|y_1)P(x_2|y_2)P(y_1)P(x_1|y_1) \\ &= \max_{y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots \max_{y_1} P(y_2|y_1)P(x_2|y_2) \text{score}_1(y_1) \\ &= \max_{y_3, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots \max_{y_2} P(y_3|y_2)P(x_3|y_3) \max_{y_1} P(y_2|y_1)P(x_2|y_2) \text{score}_1(y_1) \\ &= \max_{y_3, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots \max_{y_2} P(y_3|y_2)P(x_3|y_3) \text{score}_2(y_2) \\ &\vdots \end{aligned}$$

$$= \max_{y_n} \text{score}_n(y_n)$$



Abstract away the score for all decisions till here into **score**

Deriving the recursive algorithm

$$P(x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n) = P(y_1) \prod_{i=1}^{n-1} P(y_{i+1}|y_i) \prod_{i=1}^n P(x_i|y_i)$$

$$\max_{y_1, y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots P(y_2|y_1)P(x_2|y_2)P(y_1)P(x_1|y_1)$$

$$= \max_{y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots \max_{y_1} P(y_2|y_1)P(x_2|y_2)P(y_1)P(x_1|y_1)$$

$$= \max_{y_2, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots P(y_2|y_1)$$

$$= \max_{y_3, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots P(y_2|y_1)$$

$$= \max_{y_3, \dots, y_n} P(y_n|y_{n-1})P(x_n|y_n) \cdots P(y_2|y_1)$$

⋮

$$= \max_{y_n} \text{score}_n(y_n)$$

$$\text{score}_1(s) = P(s)P(x_1|s)$$

$$\text{score}_i(s) = \max_{y_{i-1}} P(s|y_{i-1})P(x_i|s)\text{score}_{i-1}(y_{i-1})$$

$$P(y_2|y_1)\text{score}_1(y_1)$$

Viterbi algorithm

Max-product algorithm for first order sequences

1. **Initial:** For each state s , calculate

$$\textit{score}_1(s) = P(s)P(x_1 | s)$$

2. **Recurrence:** For $i = 2$ to n , for every state s , calculate

$$\textit{score}_i(s) = \max_{y_{i-1}} P(s | y_{i-1})P(x_i | s)\textit{score}_{i-1}(y_{i-1})$$

3. **At the final state:** calculate

$$\max_{y_{i-1}} P(y, x | \pi, A, B) = \max_s \textit{score}_n(s)$$

Viterbi algorithm

Max-product algorithm for first order sequences

π : Initial probabilities

A : Transitions

B : Emissions

1. **Initial:** For each state s , calculate

$$\text{score}_1(s) = P(s)P(x_1 | s) = \pi_s B_{x_1, s}$$

2. **Recurrence:** For $i = 2$ to n , for every state s , calculate

$$\begin{aligned} \text{score}_i(s) &= \max_{y_{i-1}} P(s | y_{i-1}) P(x_i | s) \text{score}_{i-1}(y_{i-1}) \\ &= \max_{y_{i-1}} A_{y_{i-1}, s} B_{s, x_i} \text{score}_{i-1}(y_{i-1}) \end{aligned}$$

3. **At the final state:** calculate

$$\max_{y_{i-1}} P(y, x | \pi, A, B) = \max_s \text{score}_n(s)$$

Viterbi algorithm

Max-product algorithm for first order sequences

π : Initial probabilities

A : Transitions

B : Emissions

1. **Initial:** For each state s , calculate

$$\text{score}_1(s) = P(s)P(x_1 | s) = \pi_s B_{x_1, s}$$

2. **Recurrence:** For $i = 2$ to n , for every state s , calculate

$$\begin{aligned}\text{score}_i(s) &= \max_{y_{i-1}} P(s | y_{i-1}) P(x_i | s) \text{score}_{i-1}(y_{i-1}) \\ &= \max_{y_{i-1}} A_{y_{i-1}, s} B_{s, x_i} \text{score}_{i-1}(y_{i-1})\end{aligned}$$

3. **At the final state:** calculate

$$\max_{y_{i-1}} P(y, x | \pi, A, B) = \max_s \text{score}_n(s)$$

Runtime complexity:

$O(\text{sequence length} \times \text{\#possible states}^2)$

This only calculates the max. To get final answer (*argmax*):

- keep track of which state corresponds to the max at each step
- build the answer using these back pointers

From sequence labeling to syntactic parsing

Syntax: The set of principles under which sequences of words are judged to be grammatically acceptable

- We've already learned one of the most basic syntactic concepts: the syntactic role of each word
- ➡ Turning to thinking about word/phrase order

Grammar (informally) is the broader term that encompasses all implicit rules by which speakers intuitively judge which strings are well-formed and what they mean; including syntax, morphology, phonetics (sounds), semantics, and sometimes pragmatics (contextual use of language)

- Different from a **grammar formalism** that provides a set of mathematical rules or algorithms that can be used to generate the syntactic structures of a language

Syntactic parsing: The task of assigning a *syntactic structure* to a sequence of text

- Different theories of grammar propose different formalisms for describing the syntactic structure of sentences:
 - constituency grammars & dependency grammars

Why do we care?

Getting the **right interpretations of words**:

- *Visiting relatives can be annoying.*
- *Visiting relatives can be annoying.*

Gateway to thinking about recognizing **who is doing what to whom**:

- *The **cat** chased the dog.*

Machine translation from subject-verb-object (SVO) languages like English to verb-subject-object (VSO) languages like Welsh [https://en.wikipedia.org/wiki/Verb%E2%80%93subject%E2%80%93object_word_order]

Grammar checking: Sentences that cannot be parsed may have grammatical errors (or at least be hard to read)

Always useful for chunking text into phrases

Constituency parsing: Intro

Constituency parsing is a method that breaks a sentence down into its constituent parts

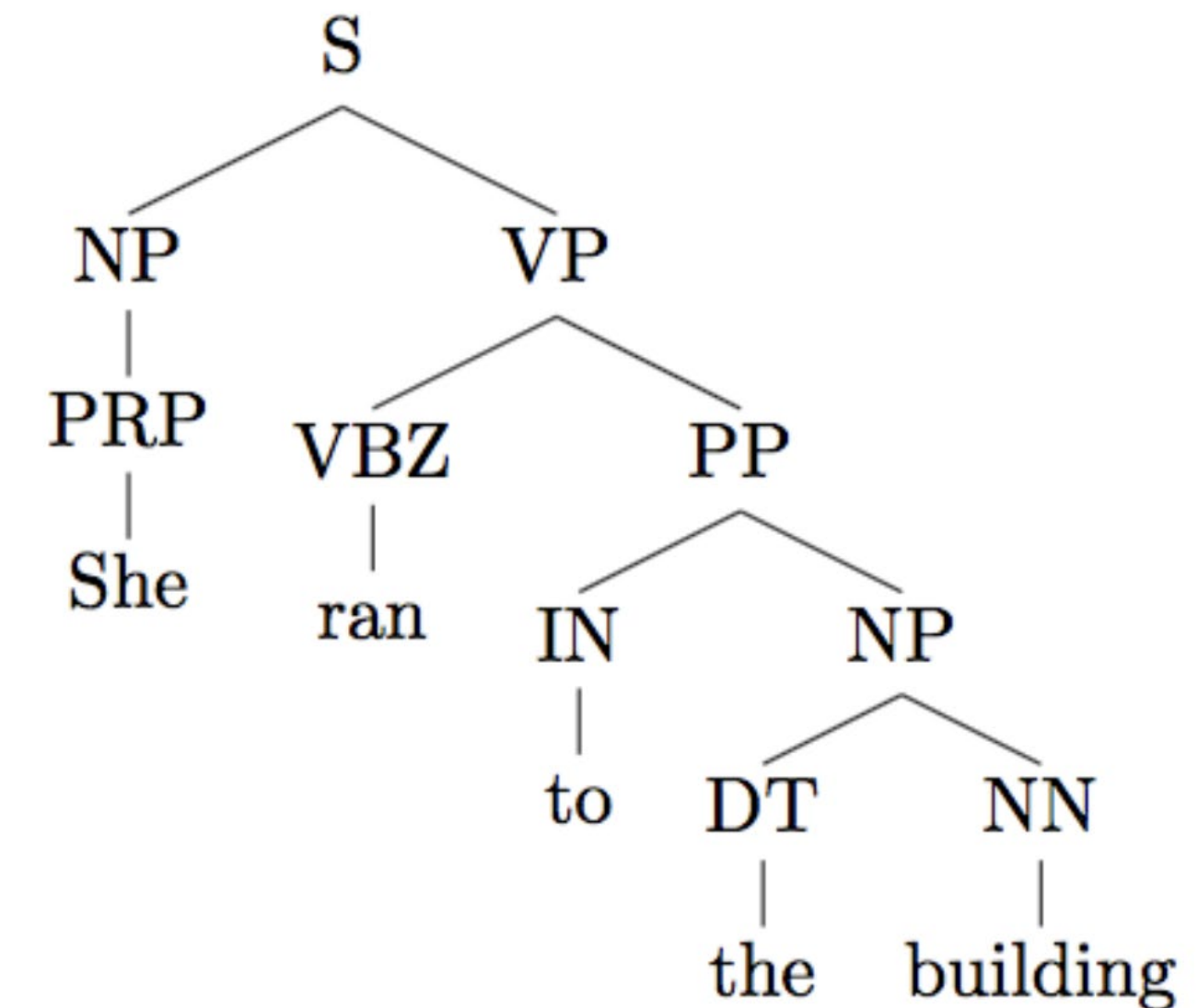
Constituents are words or groups of words that function as a single unit within a hierarchical structure

- Sentence, **N**oun **P**hrase, **V**erb **P**hrase, **P**repositional **P**hrases
- Bottom layers in POS tags

Constituents are represented in a **parse tree**

- Not a binary tree
- Right branching in English

Constituency makes sense for a lot of languages but not all, e.g., those where the word order is free such as Latin



What's hard about constituency parsing?

Structural ambiguity: When the grammar can assign more than one parse to a sentence

PP attachment ambiguity:

- *The children ate the cake with a spoon.*
- (S (NP (Det The) (N children)) (VP (V ate) (NP (Det the) (N cake)) (PP (P with) (NP (Det a) (N spoon))))))
- (S (NP (Det The) (N children)) (VP (V ate) (NP (Det the) (N cake) (PP (P with) (NP (Det a) (N spoon))))))
 - Same parse as “The children ate the cake with some icing”

Modifier scope

- *Plastic cup holder*
- (NP (Adj Plastic) (N (N Cup) (N Holder)))
- (NP (N (NP (Adj Plastic) (N Cup)) (N Holder)))

What's hard about constituency parsing? Cont.

Structural ambiguity: When the grammar can assign more than one parse to a sentence

Complement structure:

- *The students complained to the professor that they didn't understand.*
- (S
 (NP (Det The) (N students))
 (VP (V complained)
 (PP (P to) (NP (Det the) (N professor)))
 (SBAR (WHNP that) (S (NP they) (VP (V didn't) (VP understand))))))
- (S
 (NP (Det The) (N students))
 (VP (V complained)
 (PP (P to)
 (NP (Det the) (N professor)
 (SBAR (WHNP that) (S (NP they) (VP (V didn't) (VP understand)))))))

What's hard about constituency parsing? Cont.

Structural ambiguity: When the grammar can assign more than one parse to a sentence

Coordination scope:

- *I saw the man with a telescope and a hat.*

- (S
 (NP (I))
 (VP (V saw)
 (NP (Det the) (N man)
 (PP (P with)
 (NP (NP (Det a) (N telescope))
 (CC and)
 (NP (Det a) (N hat)))))))

- (S
 (NP (I))
 (VP (V saw)
 (NP (NP (Det the) (N man)
 (PP (P with)
 (NP (Det a) (N telescope))))
 (CC and)
 (NP (Det a) (N hat))))))

Let's parse!

Given a sentence, how do we find the highest scoring parse tree for it?

We'll apply the **CKY algorithm** to *Probabilistic* Context-Free Grammars

Goals of Next Lecture

Learn how to produce a constituency parse using a non-neural algorithm

- ↓ Intro
- ↓ **Context-Free Grammars (CFGs)**
- ↓ Probabilistic CFGs
- ↓ CKY Algorithm
- ↓ Evaluation